

Università Telematica eCampus

Corso di Studi Triennale in Ingegneria Informatica e dell'Automazione

**Progettazione e valutazione di un sistema di
Intrusion Detection basato su Machine Learning
per reti IoT e IIoT**

Relatore

Prof. Oleksandr Kuznetsov

Studente

De Bernardis Emanuele Pio

Matricola 001587410

A.A. 2025/2026

INDICE

Abstract

1. Introduzione

- 1.1 Contesto e motivazione
- 1.2 Problema di ricerca
- 1.3 Obiettivi e contributi della tesi
- 1.4 Struttura del lavoro

2. Stato dell'arte

- 2.1 Internet of Things e Industrial Internet of Things
 - 2.1.1 Architettura e protocolli dei sistemi IoT/IIoT
- 2.2 Sicurezza nelle reti IoT/IIoT
- 2.3 Intrusion Detection Systems
- 2.4 Machine Learning per Intrusion Detection
- 2.5 Limiti delle soluzioni esistenti
- 2.6 Posizionamento del lavoro

3. Corpo della ricerca – Risultati

- **3.1 Dataset e contesto sperimentale**
 - 3.1.1 Descrizione del dataset
 - 3.1.2 Struttura dei dati e variabili
 - 3.1.3 Definizione dei task di classificazione
 - 3.1.4 Analisi esplorativa dei dati (EDA)
 - 3.1.5 Criticità del dataset
- **3.2 Metodologia sperimentale**
 - 3.2.1 Struttura della pipeline

- 3.2.2 Preprocessing dei dati
- 3.2.3 Feature engineering
- 3.2.4 Suddivisione del dataset
- 3.2.5 Configurazione dei modelli
- 3.2.6 Metriche di valutazione
- 3.2.7 Misura dell'efficienza computazionale
- **3.3 Setup sperimentale**
- **3.4 Risultati della classificazione binaria**
- **3.5 Risultati della classificazione multiclass**
- **3.6 Analisi per classe e confronto tra modelli**
- **3.7 Discussione dei risultati**
- **3.8 Validazione statistica (k-fold)**
- **3.9 Interpretabilità (SHAP)**
- **3.10 Validazione esterna su CIC-IoT-Dataset2023**
 - 3.10.1 Motivazione e design sperimentale
 - 3.10.2 Feature engineering unificata (TON → CIC)
 - 3.10.3 Analisi del domain shift
 - 3.10.4 Risultati cross-domain
- **3.11 Baseline Deep Learning (MLP)**
 - 3.11.1 Valutazione in-domain (TON_IoT)
 - 3.11.2 Valutazione cross-domain (CIC-IoT-2023)
- **3.12 Cross-dataset generalization gap**
- **3.13 Quantizzazione e deployment embedded**

4. Conclusioni

- 4.1 Sintesi critica del lavoro
- 4.2 Limiti del lavoro
- 4.3 Sviluppi futuri
- 4.4 Implicazioni applicative

5. Bibliografia

Abstract

La crescente diffusione di infrastrutture basate su Internet of Things (IoT) e Industrial Internet of Things (IIoT) ha ampliato in modo significativo la superficie di attacco dei sistemi digitali, introducendo nuove criticità legate all'eterogeneità dei dispositivi, ai vincoli computazionali e alla crescente sofisticazione delle minacce informatiche. In tale contesto, i sistemi tradizionali di Intrusion Detection basati su firme risultano spesso insufficienti nel rilevare attacchi emergenti, varianti sconosciute e comportamenti anomali non precedentemente osservati.

La presente tesi affronta il problema del rilevamento delle intrusioni proponendo un approccio data-driven basato su tecniche di Machine Learning applicate al traffico di rete. Il lavoro si basa sull'analisi del TON_IoT Network Dataset e una validazione cross-dataset sul CIC-IoT-Dataset2023 per quantificare la portabilità del modello in presenza di domain shift, utilizzandolo per costruire una pipeline sperimentale completa, modulare e riproducibile, in grado di supportare l'intero ciclo di vita del modello, partendo dalla preparazione dei dati, all'addestramento, fino alla valutazione e al confronto tra algoritmi. La rappresentazione del traffico tramite network flows consente di descrivere le comunicazioni in forma compatta ma informativa, rendendo il problema adatto a modelli supervisionati su dati tabellari.

Il problema è formalizzabile sia come classificazione binaria, con lo scopo di distinguere tra traffico benigno e malevolo, sia come classificazione multiclass, orientata all'identificazione delle specifiche tipologie di attacco. La metodologia adottata include fasi strutturate di preprocessing, gestione delle feature, addestramento di quattro modelli supervisionati (Logistic Regression, Random Forest, XGBoost e LightGBM) e valutazione mediante metriche avanzate, tra cui F1-score, Macro-F1, ROC-AUC e PR-AUC, oltre a indicatori di efficienza quali latenza di inferenza e dimensione del modello. La robustezza statistica dei risultati è verificata mediante validazione incrociata stratificata a 5 fold. L'interpretabilità delle predizioni è analizzata tramite SHAP (SHapley Additive exPlanations) con focus sulla classe MITM, la più critica del dataset. A completamento del lavoro, i modelli migliori sono sottoposti a quantizzazione INT8 e a simulazione di deployment su hardware embedded (Arduino Mega 2560 ed ESP32-C3 tramite Wokwi).

I risultati sperimentali ottenuti mettono in evidenza prestazioni molto elevate nel task binario, con LightGBM come modello migliore sul piano predittivo ($F1 = 0.9993$) e XGBoost come scelta più equilibrata per il profilo di efficienza. Il confronto con una baseline di deep learning (MLP) conferma la superiorità degli ensemble su dati tabellari strutturati, sia in termini predittivi sia di

costo computazionale. Nel task multiclass, il sistema mostra una riduzione attesa del Macro-F1, dovuta alla maggiore complessità del problema e alla difficoltà nel riconoscimento di alcune classi minoritarie, in particolare MITM. In questo scenario, XGBoost emerge come il modello più equilibrato, grazie al miglior Macro-F1 e alla sostenibilità computazionale. Il lavoro dimostra che l'utilizzo di network flows, combinato con modelli supervisionati ben valutati, consente di sviluppare sistemi IDS efficaci, scalabili e metodologicamente solidi per contesti IoT e IIoT.

1. Introduzione

1.1 Contesto e motivazione

Negli ultimi anni la quantità di dispositivi che si servono della connessione ad internet ha subito una crescita esponenziale, determinando la diffusione capillare dell'Internet of Things (IoT). Sensori, attuatori, dispositivi intelligenti e sistemi embedded sono oggi integrati in numerosi ambiti applicativi, tra cui smart home, smart city, sanità, trasporti e automazione industriale. Questa pervasività tecnologica ha generato ecosistemi digitali di grande complessità, in cui oggetti che in passato svolgevano funzioni puramente passive sono diventati nodi attivi in grado di osservare l'ambiente, prendere decisioni elementari e cooperare con altri sistemi attraverso la rete.

Contemporaneamente, l'evoluzione verso l'Industrial Internet of Things (IIoT) ha esteso tali tecnologie ai contesti produttivi, introducendo sistemi altamente interconnessi in grado di monitorare e ottimizzare processi industriali complessi. In questi ambienti, sensori, macchine, PLC e piattaforme software sono integrati per migliorare l'efficienza operativa, ridurre i costi di manutenzione e abilitare nuovi modelli di produzione. Tuttavia l'IIoT impone requisiti significativamente più stringenti in termini di affidabilità, disponibilità e sicurezza dato che eventuali anomalie o attacchi informatici possono compromettere non solo i dati, ma anche i processi fisici e la sicurezza degli impianti.

Le reti IoT e IIoT presentano particolari vulnerabilità date dalle caratteristiche strutturali che le rendono particolarmente vulnerabili. L'eterogeneità dei dispositivi, le limitate risorse computazionali, l'utilizzo di protocolli eterogenei e l'elevato volume di traffico generato creano un terreno favorevole allo sfruttamento di queste vulnerabilità. Molti dispositivi IoT sono distribuiti in grandi quantità in ambienti difficili da controllare e vengono configurati con scarsa attenzione alla sicurezza, spesso con credenziali di default o meccanismi di autenticazione minimi. In tale contesto risultano frequenti attacchi quali DoS/DDoS, spoofing, man-in-the-middle, malware e tecniche di injection (Stallings, 2017).

I sistemi tradizionali di intrusion detection basati su firme presentano diverse limitazioni in contesti dinamici, poiché si affidano a pattern statici e non sono in grado di identificare attacchi nuovi o varianti evolute di minacce già conosciute. Per questo motivo risulta necessario adottare approcci più flessibili e guidati dai dati, capaci di apprendere direttamente dal traffico di rete e di riconoscere comportamenti anomali a prescindere dalla loro specifica forma di manifestazione. In questo contesto, gli Intrusion Detection Systems basati su Machine Learning costituiscono una soluzione particolarmente promettente, in quanto consentono di analizzare grandi quantità di traffico, riconoscere pattern complessi e adattarsi a nuove minacce senza fare affidamento esclusivo su regole statiche.

In questo nuovo scenario, la rappresentazione del traffico tramite network flows, tramite descrizioni aggregate delle comunicazioni tra dispositivi, bilancia efficacemente informatività ed efficienza computazionale, rendendo il problema adatto all'utilizzo di modelli supervisionati su dati strutturati e facilitando l'integrazione in contesti operativi reali con vincoli di latenza e risorse.

1.2 Problema di ricerca

Con il presente lavoro si intende affrontare il problema della classificazione supervisionata del traffico di rete IoT/IIoT con lo scopo di rilevare e identificare intrusioni. Il problema è formalizzato a due livelli complementari, corrispondenti a due distinte esigenze operative di un sistema IDS. Il primo livello è la classificazione binaria, che distingue traffico benigno da traffico malevolo: si tratta del filtro primario di sicurezza, utile quando l'obiettivo prioritario è individuare rapidamente la presenza di un comportamento sospetto senza dover necessariamente identificarne la natura. Il secondo livello è la classificazione multiclass, che identifica la specifica tipologia di attacco tra le categorie presenti nel dataset: questa formulazione è più complessa ma più informativa, perché consente di orientare le contromisure in modo più preciso.

1.3 Obiettivi e contributi della tesi

L'obiettivo principale è la progettazione, implementazione e valutazione di un sistema IDS prototipale basato su tecniche di Machine Learning, applicato al traffico di rete IoT/IIoT rappresentato tramite network flows.

Gli obiettivi specifici del lavoro includono l'analisi di un dataset reale di traffico IoT/IIoT TON_IoT Network Dataset basato su network flows; La progettazione e implementazione di una pipeline di preprocessing e feature engineering riproducibile; L'addestramento e il confronto di quattro modelli di Machine Learning supervisionato, con Logistic Regression, Random Forest,

XGBoost e LightGBM, sia nel task binario sia in quello multiclass; La valutazione delle prestazioni mediante metriche avanzate adatte al contesto sbilanciato; L'analisi del comportamento dei modelli per singola classe di attacco; La valutazione di aspetti operativi quali latenza di inferenza e dimensione dei modelli in ottica di deployabilità.

Il contributo principale della tesi si basa nella costruzione di un framework metodologico rigoroso, modulare e riproducibile, integrando accuratezza e predittiva, robustezza e sostenibilità computazionale, elementi da considerarsi fondamentali per l'applicazione pratica della sicurezza basata su Machine Learning in ambienti IoT e IIoT. Il lavoro non intende proporre un nuovo algoritmo, ma si prefigge il compito di dimostrare come la combinazione tra una rappresentazione adeguata del traffico, unita ad un preprocessing coerente e una valutazione attenta e articolata possa portare a risultati affidabili e soprattutto confrontabili.

1.4 Struttura del lavoro

Il documento è organizzato in quattro capitoli principali; Partendo dall'introduzione con il Capitolo 1, si continua nel Capitolo 2 con la presentazione dello stato dell'arte su IoT/IIoT, sicurezza delle reti, Intrusion Detection Systems e Machine Learning per la sicurezza, con l'obiettivo di costruire il quadro teorico necessario per motivare le scelte metodologiche e posizionare il lavoro rispetto alla letteratura esistente. Si tratterà di una selezione ragionata dei contributi più rilevanti per il problema affrontato, organizzata in modo da evidenziare i limiti delle soluzioni esistenti.

Il Capitolo 3 intende rappresentare il corpo centrale della ricerca ponendosi come il contributo più sostanziale della tesi. Infatti partendo dalla descrizione del dataset e del contesto sperimentale, procede attraverso la metodologia adottata e il setup degli esperimenti, con l'intento di presentare i risultati dei due task di classificazione binario e multiclass e per poi concludere con la discussione critica e l'interpretazione dei risultati.

Nel Capitolo 4 sono riportate le conclusioni del lavoro, strutturate in quattro parti: la sintesi critica che risponde alle domande di ricerca, l'enumerazione dei limiti riconosciuti, le direzioni di sviluppo futuro motivate dai risultati ottenuti, e le implicazioni applicative e professionali del lavoro.

La sezione bibliografica chiude il documento con i riferimenti completi alle fonti citate.

2. Stato dell'arte

2.1 Internet of Things e Industrial Internet of Things

L'Internet of Things indica un paradigma tecnologico in cui oggetti fisici dotati di sensori, capacità di elaborazione e connettività raccolgono dati, li scambiano attraverso la rete e interagiscono con altri dispositivi o con servizi remoti (Hu, 2016). In questo modo, oggetti che in passato svolgevano funzioni puramente passive diventano elementi attivi di un ecosistema digitale, capaci di osservare l'ambiente circostante, prendere decisioni elementari e cooperare con altri nodi. L'IoT ha trovato applicazione in numerosi ambiti: la domotica con sistemi di illuminazione e climatizzazione intelligente, la sanità con dispositivi indossabili e monitoraggio remoto dei pazienti, i trasporti con veicoli connessi e infrastrutture stradali intelligenti, l'agricoltura di precisione con sensori per il controllo delle colture, e le smart city con la gestione ottimizzata delle reti idriche ed energetiche.

L'Industrial Internet of Things costituisce l'estensione dell'IoT ai contesti industriali e produttivi. In questo contesto sensori, macchine, PLC, sistemi di controllo SCADA e piattaforme software sono interconnessi per monitorare e ottimizzare processi industriali, linee di produzione, infrastrutture e impianti complessi. La convergenza tra Information Technology e Operational Technology che l'IIoT rappresenta ha dato vita a nuovi modelli di produzione, come la manutenzione predittiva e la fabbrica intelligente, ma ha anche esposto a nuovi rischi sistemi che in precedenza operavano in modo isolato.

Rispetto all'IoT tradizionale, l'IIoT presenta caratteristiche più critiche. Nei sistemi industriali non è importante soltanto raccogliere e trasmettere dati, ma anche garantire continuità operativa, affidabilità, tempestività di risposta e riduzione al minimo delle interruzioni. Un malfunzionamento o un attacco informatico può avere conseguenze che vanno oltre la perdita di dati, arrivando a compromettere processi fisici, macchinari, servizi essenziali e, nei casi più gravi, la sicurezza delle persone. Un ulteriore elemento distintivo è la presenza di sistemi legacy, protocolli industriali specifici come Modbus, PROFINET e OPC-UA, e ambienti operativi fortemente eterogenei. Questo rende difficile adottare soluzioni standard di sicurezza, soprattutto quando devono essere integrate in infrastrutture già esistenti senza interrompere il funzionamento degli impianti.

2.1.1 Architettura e protocolli dei sistemi IoT/IIoT

I sistemi IoT e IIoT possono essere descritti come reti formate da dispositivi intelligenti, infrastrutture di comunicazione, servizi di elaborazione e applicazioni di supervisione. In generale, la loro architettura logica si articola su tre livelli principali. Il livello dei dispositivi fisici comprende sensori e attuatori che rilevano informazioni dall'ambiente — temperatura, pressione, movimento, consumo energetico — e inviano i dati a nodi intermedi. Il livello di comunicazione e aggregazione comprende gateway e concentratori che raccolgono i dati dai dispositivi locali, li

pre-elaborano e li trasmettono verso piattaforme di analisi. Il livello applicativo comprende le piattaforme cloud o on-premise che archiviano, elaborano e presentano i dati, supportando le applicazioni di monitoraggio e controllo (Hu, 2016).

2.2 Sicurezza nelle reti IoT/IIoT

La sicurezza è una delle principali criticità nei sistemi IoT e IIoT. L'ampia diffusione dei dispositivi, la loro eterogeneità e la limitata capacità di molte piattaforme embedded rendono difficile l'applicazione delle stesse misure di difesa adottate nei sistemi informatici tradizionali (Stallings, 2017). Uno dei problemi più rilevanti è la presenza di dispositivi con configurazioni deboli o non aggiornate, spesso esposti direttamente alla rete o protetti solo da meccanismi minimi di autenticazione. In molti casi questi dispositivi non dispongono di capacità sufficienti per eseguire operazioni di cifratura avanzata o controllo degli accessi, riducendo drasticamente il livello di protezione complessivo. I protocolli di comunicazione adottati nell'ecosistema IoT sono molto eterogenei e spesso progettati mettendo in secondo piano la sicurezza. Ad esempio possiamo notare che il livello applicativo MQTT è ampiamente utilizzato per la sua leggerezza e il modello publish/subscribe, ma nella configurazione base non include né autenticazione né cifratura. CoAP è progettato per dispositivi a risorse molto limitate e opera su UDP, con supporto opzionale per DTLS. A livello di rete locale, protocolli come Zigbee, Z-Wave e Bluetooth Low Energy gestiscono la comunicazione a corto raggio tra dispositivi, con diversi gradi di protezione. In ambito IIoT, protocolli industriali come Modbus TCP, PROFINET e OPC-UA gestiscono la comunicazione tra sistemi di controllo, con OPC-UA che offre le migliori garanzie di sicurezza ma richiede risorse computazionali più elevate.

Questa eterogeneità di protocolli ha implicazioni dirette per la sicurezza. Un sistema IDS basato su ispezione dei pacchetti dovrebbe gestire decine di protocolli diversi, molti dei quali non documentati o proprietari. La rappresentazione tramite network flows si allontana da questi dettagli poiché, invece di analizzare il payload dei singoli pacchetti, il sistema osserva le caratteristiche aggregate delle sessioni di comunicazione come volume, frequenza, durata, porte che sono informativi indipendentemente dal protocollo applicativo sottostante. Questa è una delle ragioni principali per cui l'approccio basato su flussi è particolarmente adatto al contesto IoT/IIoT.

Le reti IoT e IIoT sono esposte a una vasta gamma di minacce come ad esempio gli attacchi Denial of Service e Distributed Denial of Service che puntano a compromettere la disponibilità del servizio saturando le risorse della vittima o generando traffico eccessivo, nel contesto IoT possono colpire dispositivi e gateway, mentre in ambito IIoT possono interferire con processi industriali e servizi critici; Lo spoofing consiste nel falsificare l'identità di un dispositivo

o di un utente con l'obiettivo di ingannare il sistema o accedere a risorse protette; Il man-in-the-middle permette all'attaccante di inserirsi tra due entità comunicanti, intercettando o alterando i messaggi scambiati; Il malware comprende al suo interno software malevoli progettati per compromettere il funzionamento dei dispositivi, sottrarre informazioni o ottenere il controllo del sistema. Gli attacchi di injection, come SQL injection, sfruttano vulnerabilità nei sistemi applicativi e nei servizi esposti, consentendo l'esecuzione di codice malevolo o la manipolazione di dati.

La protezione dei sistemi IoT e IIoT non può essere esclusivamente basata su difese perimetrali o controlli statici, ma è necessario affiancargli strumenti capaci di monitorare il comportamento della rete che identifichino tempestivamente eventuali deviazioni dal normale funzionamento. In questo contesto, gli Intrusion Detection Systems rappresentano una componente essenziale dell'architettura di sicurezza.

2.3 Intrusion Detection Systems

Un Intrusion Detection System è un componente di sicurezza informatica progettato per monitorare il comportamento di una rete o di un sistema e individuare attività potenzialmente malevole, tentativi di accesso non autorizzato o deviazioni significative rispetto al funzionamento atteso (Stallings, 2017). A differenza di un firewall, che opera in modo preventivo bloccando il traffico indesiderato, un IDS svolge principalmente una funzione di osservazione e rilevamento: quando viene identificato un evento sospetto, il sistema può generare un allarme, registrare l'evento per analisi successive o, nei sistemi più evoluti denominati Intrusion Prevention Systems (IPS), attivare meccanismi di risposta automatica.

Dal punto di vista architetturale, gli IDS si suddividono in diverse categorie. Gli Host-based IDS sono installati su singoli dispositivi e analizzano risorse locali come log di sistema, processi in esecuzione, modifiche ai file e attività degli utenti. Questa caratteristica li rende efficaci nel rilevare compromissioni a livello di host, ma ne riduce l'utilizzo in contesti IoT/IIoT, dove le risorse computazionali disponibili sono spesso limitate. I Network-based IDS focalizzano l'attenzione sul traffico di rete, monitorando le interazioni tra i vari nodi, questo approccio è particolarmente indicato quando l'obiettivo è controllare l'intera infrastruttura senza intervenire sui singoli dispositivi, risultando quindi il più rilevante per il presente lavoro. I Distributed IDS combinano più nodi di osservazione distribuiti nella rete per migliorare la visibilità globale, ma introducono anche una maggiore complessità nella gestione e nella sincronizzazione delle informazioni.

Le tecniche di rilevamento delle intrusioni possono essere suddivise in quattro famiglie principali, come la signature-based detection utilizza un insieme di pattern già noti per riconoscere attacchi già osservati in precedenza ed è molto efficace contro minacce catalogate producendo generalmente pochi falsi positivi quando le firme sono ben definite, ma non è in grado di identificare attacchi nuovi o varianti non ancora catalogate. La anomaly-based detection costruisce invece un modello del comportamento normale del sistema e segnala qualsiasi deviazione significativa, questa strategia è utile per rilevare attacchi sconosciuti o zero-day, ma può produrre un numero più elevato di falsi positivi in ambienti dinamici.

La specification-based detection si pone lo scopo di verificare il rispetto di regole specifiche di comportamento predefinito, offrendo un buon compromesso tra precisione e generalizzazione a sottintendendo vi sia una conoscenza approfondita del sistema osservato. Gli approcci ibridi, invece, combinano più tecniche con l'obiettivo di sfruttarne i rispettivi vantaggi introducendo però una maggiore complessità a livello implementativo.

Nel contesto delle reti IoT e IIoT, la progettazione di un IDS deve tenere conto di specifici vincoli: la limitata capacità computazionale dei dispositivi, la distribuzione geografica dei nodi, l'eterogeneità dei protocolli e la necessità di garantire un'elevata disponibilità impongono l'adozione di soluzioni efficienti, scalabili e capaci di operare con bassa latenza; in questo contesto, l'analisi basata sui network flows si dimostra particolarmente adatta, poiché impiega rappresentazioni aggregate che descrivono le comunicazioni in modo sintetico, garantendo un equilibrio tra contenuto informativo e costo computazionale.

Un flusso di rete è tipicamente definito come la sequenza di pacchetti che condividono gli stessi valori per un insieme di chiavi — tipicamente la quintupla composta da indirizzo IP sorgente, indirizzo IP destinazione, porta sorgente, porta destinazione e protocollo — in un dato intervallo di tempo. Le feature estratte da un flusso includono statistiche di volume (byte e pacchetti trasmessi nelle due direzioni), statistiche temporali (durata, inter-arrival time dei pacchetti), informazioni sullo stato della connessione (flag TCP osservati, stato della sessione) e caratteristiche dei protocolli applicativi osservati (tipo di query DNS, metodo HTTP, versione SSL). Questa rappresentazione tabellare strutturata si presta naturalmente all'utilizzo di modelli di classificazione supervisionata su dati tabellari, senza richiedere pipeline di analisi sequenziale come quelle necessarie per dati di testo o immagini.

L'integrazione di un IDS che si basa sui network flows in un'architettura IoT/IIoT può avvenire a diversi livelli. A livello di gateway, che è il punto di raccolta del traffico tra la rete locale IoT e la rete esterna, è possibile installare un motore di analisi dei flussi che monitora tutte

le comunicazioni in ingresso e in uscita. Questo approccio centralizzato è efficiente poiché permette di osservare il traffico per intero da un singolo punto di analisi, però richiede che il gateway abbia risorse computazionali adeguate. In alternativa, in architetture distribuite, possono essere installati moduli di analisi più leggeri a livello di segment, formati da gruppi di dispositivi IoT omogenei, con un coordinatore centrale che aggrega le segnalazioni e gestisce la risposta agli incidenti. In entrambi i casi, la rappresentazione tramite flussi riduce in maniera significativa il volume di dati da processare rispetto all'analisi a livello di pacchetto, riducendo in questo modo i requisiti di banda e di capacità elaborativa del sistema di monitoraggio.

2.4 Machine Learning per Intrusion Detection

Il Machine Learning è un insieme di tecniche che consente ai sistemi informatici di apprendere dai dati e migliorare le proprie capacità di previsione o classificazione senza essere programmati esplicitamente per ogni singolo caso (Bishop, 2006). Nell'apprendimento supervisionato, paradigma adottato in questa tesi, il modello viene addestrato su un dataset etichettato in cui a ogni osservazione è associata una classe di appartenenza. L'obiettivo è costruire una funzione che, a partire dalle feature di input, sia in grado di prevedere correttamente la classe corrispondente per nuove osservazioni non viste durante il training. Nel caso dell'Intrusion Detection, questo approccio è particolarmente adatto quando si dispone di dati di traffico già classificati come benigni o malevoli, o etichettati con la tipologia specifica di attacco.

I modelli di Machine Learning presentano diversi vantaggi rispetto agli approcci tradizionali basati su firme. In primo luogo, possono generalizzare beyond le firme note e rilevare comportamenti sospetti anche quando non corrispondono a un pattern predefinito. In secondo luogo, consentono di gestire grandi quantità di dati e di individuare relazioni complesse difficili da formalizzare manualmente. In terzo luogo, se addestrati su dati rappresentativi, possono adattarsi a variazioni nel traffico senza richiedere aggiornamenti manuali delle regole. Questi vantaggi li rendono particolarmente interessanti per scenari IoT/IIoT, dove la varietà e la dinamicità del traffico rendono impraticabile la definizione manuale di tutte le firme di attacco possibili (Géron, 2019).

Per il problema affrontato in questa tesi — classificazione supervisionata su dati tabellari derivati da network flows — sono stati selezionati quattro modelli con caratteristiche complementari, come riportato nella tabella seguente.

Modello	Tipo	Vantaggi principali	Limiti principali
---------	------	---------------------	-------------------

Logistic Regression	Lineare	Veloce, interpretabile, baseline robusta	Non cattura relazioni non lineari
Random Forest	Ensemble (bagging)	Robusto, gestisce non linearità, stabile su dati tabellari	Pesante in memoria, lento in inferenza
XGBoost	Ensemble (boosting)	Alta accuratezza, efficiente, buon controllo overfitting	Richiede tuning attento degli iperparametri
LightGBM	Ensemble (boosting)	Molto veloce nel training, efficiente su dataset grandi	latenza alta nel task multiclass (621 ms/1k) a causa della crescita leaf-wise

Tabella 2.1 – Modelli di Machine Learning utilizzati e relative caratteristiche

La Logistic Regression è il modello lineare di riferimento: stima la probabilità che un'osservazione appartenga a una certa classe attraverso una funzione logistica. È veloce sia in addestramento sia in inferenza, molto interpretabile grazie ai coefficienti espliciti, e rappresenta una baseline importante per misurare il guadagno introdotto da modelli più complessi. La sua semplicità è anche il suo principale limite: in presenza di relazioni fortemente non lineari o interazioni complesse tra feature, le sue prestazioni possono risultare inferiori rispetto ad approcci più flessibili.

La Random Forest è una tecnica ensemble che costruisce molti alberi decisionali, ognuno addestrato su campioni casuali di dati e sottoinsiemi di variabili, combinando poi le loro predizioni attraverso un meccanismo di voto a maggioranza. Questo metodo è pensato per limitare l'overfitting rispetto a un singolo albero, riuscendo al contempo a catturare relazioni non lineari e interazioni tra le feature senza necessitare di trasformazioni esplicite dei dati. Si dimostra particolarmente adatto ai dati tabellari e presenta una buona robustezza nei confronti di outlier e feature ridondanti, sebbene questo comporti un maggiore utilizzo di memoria e una latenza di inferenza più alta rispetto ai modelli basati su boosting.

Il Gradient Boosting costruisce una sequenza di modelli deboli correggendo gradualmente gli errori dei modelli precedenti. Questo approccio iterativo permette di ottenere prestazioni molto elevate su dati strutturati. XGBoost adotta regolarizzazione L1 e L2, gestione nativa dei valori mancanti e parallelizzazione efficiente; LightGBM, invece, adotta una strategia di crescita degli alberi leaf-wise anziché level-wise, questo ci consente di gestire dataset di grandi dimensioni con minor consumo di memoria e tempi di training ridotti. Entrambi sono particolarmente interessanti per applicazioni IoT/IIoT grazie al buon compromesso tra accuratezza e costo computazionale.

La valutazione dei modelli necessita di un insieme di metriche complementari, dove l'accuracy misura la proporzione di predizioni corrette, ma può essere fuorviante in presenza di dataset sbilanciati; La precision indica la quota di predizioni positive effettivamente corrette,

riducendo i falsi allarmi; Il recall misura la capacità del modello di identificare tutte le istanze positive, che è un aspetto critico nell'ambito della sicurezza, dove un attacco non rilevato può avere conseguenze gravi.

L'F1-score, che è la metrica più importante, rappresenta la media armonica tra precision e recall. Il Macro-F1 calcola l'F1 per ciascuna classe e ne fa la media non ponderata, assegnando lo stesso peso a tutte le categorie indipendentemente dalla loro frequenza, è la metrica principale per valutare il comportamento in presenza di class imbalance. I ROC-AUC e PR-AUC, invece misurano la capacità discriminativa al variare della soglia di decisione, con PR-AUC particolarmente informativa quando la classe positiva è rara.

Oltre all'apprendimento supervisionato, esistono altri paradigmi rilevanti per il problema IDS. Ad esempio l'apprendimento non supervisionato analizza dati privi di etichette con l'obiettivo di individuare strutture nascoste o anomalie, utilizzando algoritmi come k-means, DBSCAN e Isolation Forest possono identificare flussi di rete anomali senza richiedere dati etichettati per tutte le possibili minacce. Questo approccio è di particolare interesse per la rilevazione di attacchi zero-day o varianti non ancora catalogate. Tuttavia, richiede una calibrazione più attenta e può produrre risultati meno immediati rispetto ai modelli supervisionati, specialmente quando tra il comportamento normale e anomalo non vi sono differenze molto nette (Bishop, 2006).

L'apprendimento semi-supervisionato combina dati etichettati e non etichettati: utile quando le etichette sono disponibili solo in parte o quando il costo della labellazione è elevato, può essere interessante in scenari reali dove il traffico etichettato è limitato ma il traffico non etichettato è abbondante. Nel presente lavoro viene adottato esclusivamente l'approccio supervisionato, in quanto il TON_IoT fornisce etichette complete per tutti i flussi.

2.5 Limiti delle soluzioni esistenti

La letteratura sugli IDS per IoT e IIoT ha mostrato progressi significativi negli ultimi anni, ma anche limiti ricorrenti che motivano la direzione del presente lavoro. Un aspetto critico, che viene spesso trascurato nella letteratura IDS è la robustezza degli approcci ML ad attacchi avversari. Un attaccante sofisticato conoscendo l'esistenza di un sistema IDS basato su ML potrebbe modificare volontariamente le caratteristiche del proprio traffico per eludere la classificazione, ad esempio suddividendo un attacco DoS in flussi di dimensione più piccola abbassando i byte per flusso sotto la soglia critica, o ritardando artificialmente i pacchetti rendendo la durata della connessione compatibile con il traffico legittimo, questa vulnerabilità è chiamata adversarial ML applicato ai sistemi IDS.

Un primo limite riguarda la qualità e la rappresentatività dei dataset impiegati: molti studi si basano su dataset generati in ambienti controllati o con traffico sintetico, ottenendo risultati molto elevati in fase sperimentale ma difficilmente trasferibili a contesti reali.

Nei dataset simulati, i pattern degli attacchi tendono a essere più netti e facilmente distinguibili rispetto a quelli osservabili nel traffico reale, dove la variabilità dei comportamenti legittimi e la presenza di rumore rendono la classificazione più complessa. Questa differenza tra risultati ottenuti in ambiente controllato e prestazioni in scenari operativi costituisce uno dei principali divari tra la ricerca accademica e l'adozione concreta dei sistemi IDS basati su Machine Learning.

Un secondo limite ricorrente riguarda la scelta delle metriche. Molti lavori valutano le prestazioni quasi esclusivamente attraverso l'accuracy, che in presenza di classi sbilanciate può risultare fuorviante: un classificatore che predice sempre la classe maggioritaria può ottenere accuracy elevata pur avendo recall nullo per le classi di attacco rare. L'adozione sistematica di metriche più robuste come Macro-F1, PR-AUC e analisi per classe è ancora relativamente rara, specialmente nei lavori più orientati all'ingegneria. Questa lacuna è particolarmente grave in sicurezza informatica: le classi di attacco rare non sono necessariamente meno pericolose di quelle frequenti — anzi, attacchi sofisticati come MITM o APT (Advanced Persistent Threats) tendono a essere meno frequenti proprio perché richiedono più risorse e capacità da parte dell'attaccante, ma quando si verificano possono avere conseguenze molto più gravi (Alsaedi et al. (2020)).

Un terzo aspetto critico è la scarsa attenzione alla spiegabilità poiché la maggior parte dei lavori si concentra sulla qualità predittiva dei modelli, trascurando aspetti operativi come ad esempio la latenza di inferenza, dimensione del modello, consumo di memoria e integrazione con infrastrutture di rete reali. In ambienti IoT e IIoT, dove le risorse computazionali sono limitate e la rapidità della risposta è fondamentale, questi aspetti sono altrettanto rilevanti delle metriche di classificazione poiché un modello con accuracy marginalmente superiore ma latenza dieci volte più alta o dimensione venti volte maggiore può essere significativamente meno utile in un contesto operativo con gateway IoT da pochi GB di RAM. La valutazione comparativa di efficienza operativa affiancata alle metriche predittive è uno degli aspetti che il presente lavoro intende valorizzare rispetto alla letteratura esistente.

Infine, la spiegabilità delle decisioni resta un tema aperto che la letteratura riconosce come importante ma spesso non affronta in modo sistematico. In un contesto di sicurezza, non basta sapere che un evento è classificato come sospetto è necessario comprenderne il perché, per permettere all'analista umano di validare la decisione, agire in modo informato e, eventualmente, migliorare il sistema identificando falsi positivi sistematici. La mancanza di spiegabilità è anche

una barriera all'adozione dei sistemi ML in contesti regolamentati, dove le decisioni automatizzate devono essere giustificabili e auditabili. Questi limiti definiscono il perimetro all'interno del quale si posiziona il presente lavoro e motivano le scelte metodologiche adottate.

2.6 Posizionamento del lavoro

Alla luce di quanto emerso, il presente lavoro si inserisce nella linea di ricerca degli IDS supervisionati su traffico di rete, con focus su ambienti IoT/IIoT, valutazione binaria e multiclass, e attenzione non solo alle metriche classiche ma anche a robustezza sulle classi minoritarie, latenza di inferenza e deployabilità. La scelta di operare su network flows in formato tabellare è coerente con la letteratura più recente, che mostra come questa rappresentazione bilanci efficacemente informatività e costo computazionale (Han et al., 2012; Géron, 2019).

La scelta di modelli tabellari consolidati come in questo caso con Logistic Regression, Random Forest, XGBoost e LightGBM è stata effettuata prendendo in osservazione la natura strutturata del problema mantenendo come obiettivo la confrontabilità e interpretabilità dei risultati. Il lavoro adotta modelli supervisionati robusti e ben compresi, valutandoli in modo sistematico su due formulazioni.

Il contributo centrale di questo lavoro consiste nella progettazione di un framework metodologico rigoroso e facilmente trasferibile, cioè una pipeline end-to-end che integra accuratezza predittiva, robustezza e sostenibilità computazionale. In questo modo viene messo in evidenza come questi tre elementi debbano essere valutati congiuntamente per ottenere un IDS realmente efficace negli ambienti IoT e IIoT.

3. Corpo della ricerca – Risultati

3.1 Dataset e contesto sperimentale

3.1.1 Descrizione del dataset

Il dataset utilizzato è il TON_IoT Network Dataset (Moustafa & Slay, 2020), un riferimento consolidato nella comunità IDS per la qualità realistica del traffico che contiene. Il dataset è stato prodotto in un ambiente di test che replica una rete IoT e IIoT composta da dispositivi eterogenei come sensori di temperatura, umidità, movimento e pressione, telecamere intelligenti, sistemi di controllo industriale, inseriti in un contesto che combina traffico legittimo e sessioni di attacco realizzate da ricercatori di sicurezza. I flussi di rete sono stati raccolti con strumenti specializzati come Argus e Zeek (precedentemente noto come Bro), che permettono di ottenere rappresentazioni strutturate delle comunicazioni a livello di flusso aggregato.

La decisione di utilizzare i network flows deriva da motivazioni sia operative che metodologiche. Sul piano operativo, l'analisi a livello di pacchetto produce una quantità di dati molto più elevata e comporta un forte consumo di risorse computazionali, risultando quindi poco compatibile con i vincoli tipici degli ambienti IoT e IIoT. I flussi aggregati, al contrario, rappresentano le comunicazioni in forma compatta tramite statistiche di sessione, conservando comunque un'informazione sufficiente per discriminare tra attività lecite e comportamenti malevoli. Dal punto di vista metodologico, i dati tabellari ottenuti dai network flows sono particolarmente adatti all'impiego di modelli di Machine Learning supervisionato, poiché operano su feature numeriche e categoriali ben strutturate senza la necessità di pipeline di preprocessing particolarmente complesse.

3.1.2 Struttura dei dati e variabili

Il file di lavoro principale è un dataset tabellare in cui ogni riga corrisponde a un singolo flusso di rete e ogni colonna rappresenta una caratteristica del traffico. Nella configurazione iniziale, il dataset contiene 211.043 flussi e 44 colonne complessive. Tra le feature disponibili figurano informazioni di rete di base come indirizzo IP sorgente e destinazione, porte sorgente e destinazione, protocollo e servizio. Le feature di flusso includono la durata della connessione, il numero di byte trasmessi e ricevuti, il numero di pacchetti nelle due direzioni, il volume di byte a livello IP sorgente e destinazione e i byte persi. Le feature di protocollo superiore comprendono informazioni relative a DNS come, tipo di query, classe, codice di risposta, HTTP come, metodo, URI, lunghezza del body, codice di stato e SSL nel contesto di, versione, cifrario, stato della sessione.

Il dataset mette a disposizione due variabili target distinte. La variabile *label* è utilizzata per la classificazione binaria, in cui 0 rappresenta il traffico normale e 1 indica il traffico malevolo. La variabile *type*, invece, è destinata alla classificazione multiclasse, assegnando ogni flusso a una specifica categoria, tra cui normal, ddos, dos, scanning, injection, xss, ransomware, password, backdoor e mitm.

3.1.3 Definizione dei task di classificazione

Il problema di Intrusion Detection è formalizzato a due livelli complementari che corrispondono a due distinte esigenze operative. Nel task binario, il modello deve stabilire se un flusso di rete appartiene alla classe normale oppure alla classe di attacco: si tratta del livello più immediato di rilevamento, utile come primo filtro di sicurezza. Dal punto di vista operativo, questa formulazione permette di identificare rapidamente la presenza di un comportamento sospetto senza dover ancora determinarne la natura specifica, il che è particolarmente utile in

scenari dove la velocità di risposta è prioritaria rispetto alla precisione diagnostica. Il target binario è la variabile label, con codifica 0 per il traffico normale e 1 per il traffico di attacco.

Nel task multiclass, il sistema deve assegnare ogni flusso a una categoria precisa tra quelle presenti nel dataset. Questa formulazione è più complessa ma più informativa poiché ci consente di comprendere non solo se la rete sia sotto attacco, ma anche quale tecnica stia venendo utilizzata, fornendo informazioni essenziali per orientare le contromisure nel modo più appropriato. Un alert che identifica un attacco come ransomware richiede una risposta diversa rispetto a uno scanning o a un attacco di tipo password. Per questo motivo la capacità di discernere tra le varie tipologie di attacco trasforma il sistema da semplice rilevatore a strumento fondamentale per il supporto alla decisione di operatori di sicurezza.

La presenza di entrambi i task nella medesima pipeline ha un valore metodologico implementativo poiché ci consente di confrontare in maniera precisa le prestazioni dei modelli in due scenari di difficoltà crescente e di analizzare come il Macro-F1, che è la metrica principale, vari tra i due task. Questa comparazione diretta ci restituisce una misura quantitativa dell'incremento di difficoltà introdotto dalla classificazione permettendoci di identificare quali modelli presentano performance migliori all'aumentare della complessità del problema.

3.1.4 Analisi esplorativa dei dati (EDA)

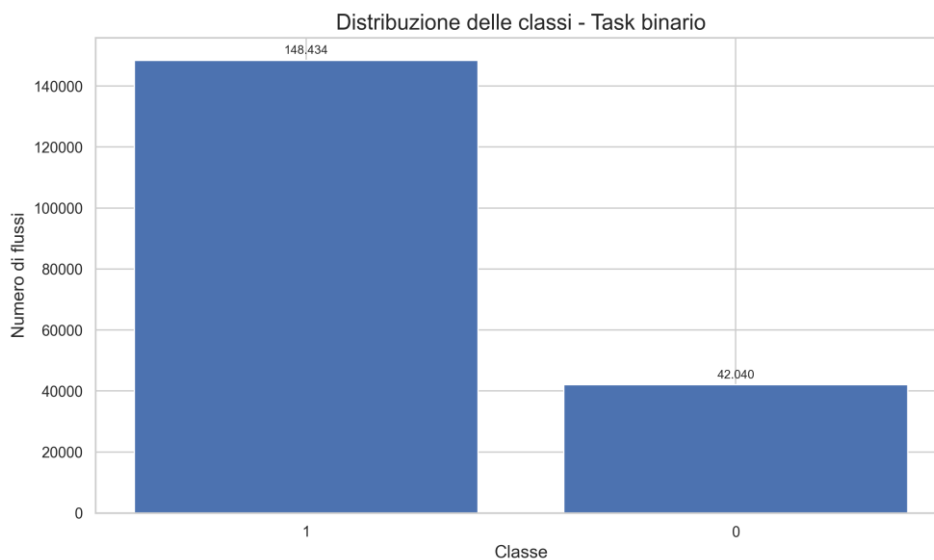


Figura 3.1 – Distribuzione delle classi nel task binario del TON_IoT Network Dataset.

La distribuzione delle classi nel problema binario evidenzia uno sbilanciamento rilevante: la classe di attacco (label = 1) comprende 148.434 flussi, mentre il traffico normale (label = 0) ne

include 42.040, con un rapporto di circa 3,5 a 1. Sebbene questo sbilanciamento sia meno marcato rispetto a quello osservato in molti dataset IDS reali, esso influisce direttamente sulla selezione delle metriche di valutazione.

Nel task multiclass il quadro è più articolato, come mostrato nella tabella seguente.

Classe (type)	N. flussi	% sul totale	Categoria
normal	42.040	22.1%	Traffico legittimo
scanning	20.000	10.5%	Attacco
ddos	19.993	10.5%	Attacco
injection	19.964	10.5%	Attacco
password	19.861	10.4%	Attacco
dos	18.992	10.0%	Attacco
backdoor	18.711	9.8%	Attacco
xss	15.137	7.9%	Attacco
ransomware	14.735	7.7%	Attacco
mitm	1.041	0.6%	Attacco (classe rara)

Tabella 3.1 – Distribuzione delle classi nel task multiclass (TON_IoT Network Dataset)

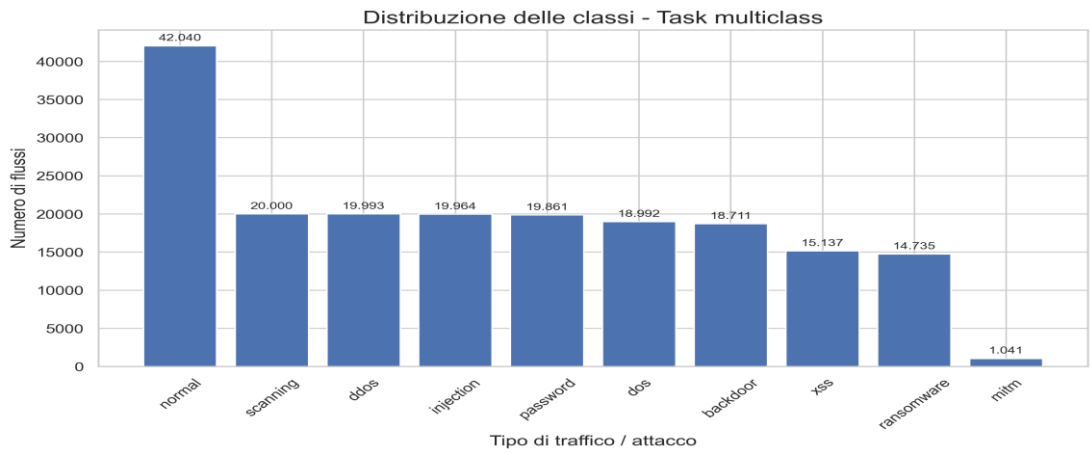


Figura 3.2 – Distribuzione delle classi nel task multiclass del TON_IoT Network Dataset.

La classe mitm, con soli 1.041 campioni pari allo 0,6% del totale, è nettamente la meno rappresentata e costituisce il caso più critico per l'intera pipeline. Questa classe è caratterizzata da una complessità semantica significativa poiché un attacco Man-in-the-Middle lascia tracce nel traffico di rete meno evidenti rispetto a un DoS, che genera volumi anomali, o a uno scanning, che produce pattern di connessione caratteristici, tutto ciò rende MITM la sfida principale del task multiclass.

L'analisi dei valori mancanti evidenzia la presenza di numerose colonne caratterizzate da alte percentuali di dati assenti. In particolare, le feature legate a HTTP, come *http_orig_mime_types*, *http_resp_mime_types*, *http_method*, *http_user_agent* e *http_trans_depth*, presentano valori mancanti in oltre il 90% dei flussi, dal momento che gran parte del traffico IoT non fa uso del protocollo HTTP. Anche le feature relative a SSL mostrano un comportamento simile. Questo fenomeno è del tutto atteso in un dataset di network flows IoT/IIoT, dove protocolli applicativi come HTTP e SSL non sono presenti in tutte le tipologie di comunicazione.

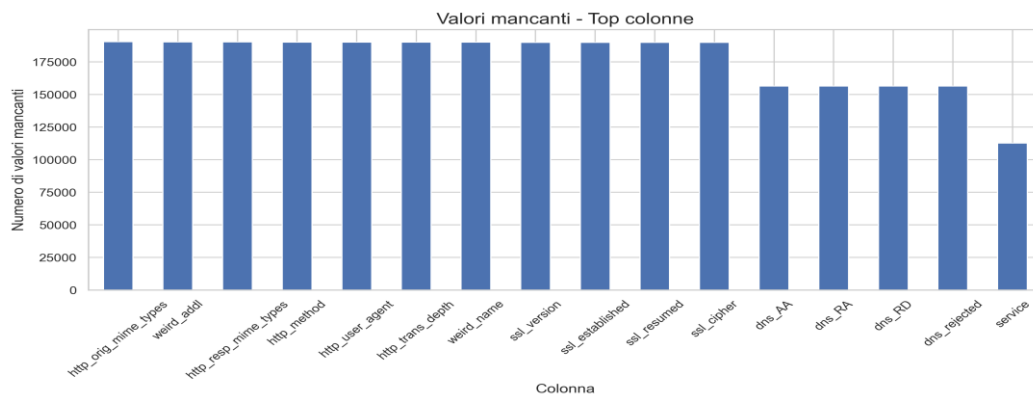


Figura 3.3 – Principali colonne con valori mancanti nel TON_IoT Network Dataset.

La matrice di correlazione tra le variabili numeriche ha fatto emergere cluster di correlazione elevata tra feature affini come *src_bytes*, *src_pkts* e *src_ip_bytes* che sono fortemente correlati, così come *dst_bytes*, *dst_pkts* e *dst_ip_bytes*. Questa ridondanza informativa è gestita dal preprocessing attraverso la rimozione di feature ad alta cardinalità e la standardizzazione stabilizzando la scala delle feature numeriche correlate senza eliminarne esplicitamente nessuna.

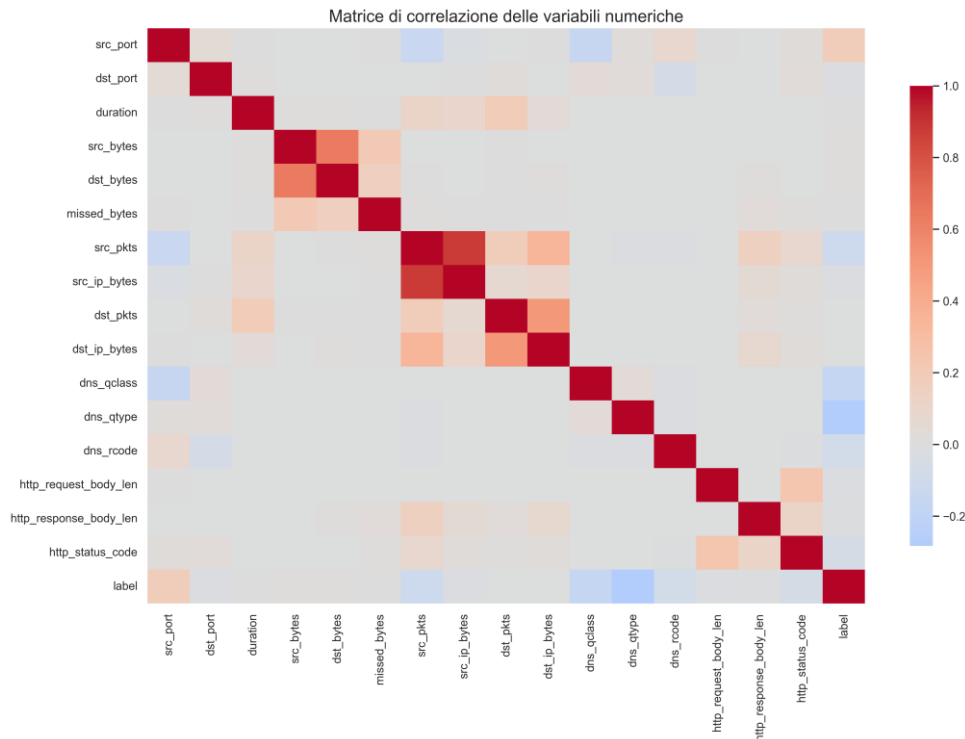


Figura 3.4 – Matrice di correlazione tra le variabili numeriche del dataset.

3.1.5 Criticità del dataset

Il dataset presenta alcune criticità significative che possono influenzare l'interpretazione dei risultati e che devono essere esplicitate prima della fase di modellazione.

La prima riguarda lo sbilanciamento delle classi, che impone l'utilizzo di metriche appropriate per dati non bilanciati e rende opportuno considerare, in sviluppi futuri, tecniche di campionamento per migliorare la capacità del modello di riconoscere le classi meno rappresentate.

La seconda criticità riguarda la generalizzabilità. Il TON_IoT è costruito in un ambiente di test controllato con traffico semi-sintetico, cioè i pattern di attacco sono generati da ricercatori di sicurezza in condizioni di laboratorio, il che rende le signature degli attacchi più nette e meno variabili rispetto al traffico prodotto da attaccanti reali che adattano le proprie tecniche per eludere la rilevazione. Questo può produrre valori di performance ottimistici rispetto a ciò che si otterrebbe su traffico reale acquisito in produzione. Non si tratta di un limite specifico di questo dataset ma è una caratteristica comune a quasi tutti i dataset IDS disponibili pubblicamente.

La terza criticità riguarda la presenza di feature ad alta cardinalità e colonne con percentuali molto elevate di valori mancanti come già espresso nell'analisi esplorativa, che se

mantenuti, potrebbero introdurre data leakage, portando il modello ad imparare a riconoscere il traffico di attacco dai valori specifici degli URI o dei certificati usati nell'esperimento, non dai pattern generali del traffico. La rimozione di queste colonne nel preprocessing è una scelta metodologicamente necessaria, ma comporta anche la perdita di informazioni potenzialmente utili in un deployment reale dove tali feature potrebbero essere disponibili e significative.

3.2 Metodologia sperimentale

3.2.1 Struttura della pipeline

La pipeline sviluppata segue un approccio modulare e riproducibile, articolato in cinque fasi sequenziali:

preparazione	e	pulizia	dei	dati;
analisi				esplorativa;
preprocessing	e	feature		engineering;
addestramento		dei		modelli;
valutazione		e		confronto;

Questa struttura garantisce coerenza tra le scelte teoriche e l'implementazione pratica, permettendo di eseguire ogni fase in modo indipendente facilitando la verifica e la replicazione dei risultati.

La separazione tra preparazione, preprocessing, training e valutazione non è soltanto una soluzione organizzativa, ma una precisa decisione progettuale. In particolare, garantisce che i trasformatori di preprocessing siano fittati esclusivamente sui dati di training e applicati ai dati di test senza mai trasferire informazione tra le due partizioni, condizione essenziale per evitare data leakage. L'intera pipeline è implementata tramite le classi Pipeline e ColumnTransformer di scikit-learn, che impongono questa separazione in modo strutturale.

3.2.2 Preprocessing dei dati

La fase di preprocessing ha inizio con la normalizzazione dei placeholder testuali comuni: i valori "-", "NA", "N/A" e le stringhe vuote vengono trattati come valori mancanti (NaN), garantendo coerenza nella gestione dei missing values nelle fasi successive. Si procede quindi all'identificazione e alla rimozione delle colonne non utili alla modellazione. Vengono eliminate le colonne di identificazione dei dispositivi — src_ip e dst_ip — poiché contengono indirizzi IP specifici dell'ambiente di test che non sarebbero generalizzabili ad altri contesti e potrebbero introdurre data leakage. Vengono rimosse le colonne testuali ad alta cardinalità — http_uri, ssl_subject, ssl_issuer, dns_query, http_host, user_agent — perché il numero elevato di valori unici genererebbe sparsità nella codifica e ridurrebbe la robustezza del modello. Le colonne

costanti, con un solo valore unico in tutto il dataset, vengono anch'esse eliminate. Infine, si procede alla rimozione dei duplicati esatti.

Il preprocessing numerico prevede due passaggi sequenziali. Il primo è l'imputazione con mediana, poichè la mediana non è influenzata da valori estremi, che sono comuni nei dati di traffico di rete, dove un singolo flusso anomalo può avere byte o durata enormi, e fornendo una stima più stabile del valore centrale.

Il secondo aspetto riguarda l'applicazione dello *StandardScaler*, che normalizza ciascuna variabile rendendola a media zero e deviazione standard unitaria. Questa operazione è fondamentale per assicurare la corretta convergenza della Logistic Regression, che è influenzata dalla scala delle feature, mentre non ha effetti sui modelli basati su alberi decisionali, i quali sono invarianti rispetto a trasformazioni monotone delle variabili.

Il preprocessing categorico prevede imputazione alla moda, ossia il valore più frequente, seguita da One-Hot Encoding, che converte ogni variabile categorica in un insieme di variabili binarie, una per ogni categoria. L'opzione `handle_unknown="ignore"` garantisce che le categorie non viste durante il training non causino errori in inferenza, comportamento rilevante in un contesto di deployment reale dove il traffico può contenere valori non presenti nel dataset di addestramento. Il threshold di cardinalità per la selezione delle feature categoriche è fissato a 50 valori unici, le feature con cardinalità superiore considerate ad alta cardinalità vengono rimosse nella fase di preparazione. Dopo il preprocessing, il dataset si riduce da 211.043 righe e 44 colonne a 190.474 righe e 36 colonne.

3.2.3 Feature engineering

Il feature engineering adottato è volutamente conservativo: non vengono introdotte feature derivate o trasformazioni artificiali aggiuntive rispetto a quelle già presenti nel dataset originale. Questa scelta è motivata dall'obiettivo di costruire un sistema il più possibile generalizzabile: in un contesto operativo, feature costruite su misura per un dataset specifico, come ratios calcolati su coppie di colonne o aggregazioni temporali che sfruttano la struttura specifica del dataset, rischiano di non essere disponibili su traffico reale con una struttura diversa, o di comportarsi in modo imprevedibile quando la distribuzione del traffico cambia nel tempo.

La scelta conservativa è coerente con la letteratura poichè sappiamo che, per dati tabellari ben strutturati provenienti da network flows, la qualità del preprocessing tende ad avere un impatto maggiore sulle prestazioni rispetto alla complessità del feature engineering. In altre parole, rimuovere feature ridondanti o fuorvianti, gestire correttamente i valori mancanti e standardizzare le scale contribuisce al miglioramento delle prestazioni rispetto all'aggiunta di

nuove feature derivate. Questo principio è particolarmente rilevante in presenza di modelli ensemble come Random Forest e XGBoost, che sono capaci attraverso i meccanismi di feature importance interni di selezionare le feature più informative durante il training e di ignorare quelle meno rilevanti.

Per la validazione esterna presentata nella Sezione 3.10, la pipeline di feature engineering viene ampliata introducendo uno schema di trasformazione condiviso, che permette di mappare dataset eterogenei in un unico spazio delle feature comune.

Lo schema unificato comprende 10 variabili numeriche ottenute da statistiche aggregate dei flussi, tra cui volumi di byte e pacchetti, asimmetrie direzionali, durata e frequenza, oltre a 3 variabili categoriche armonizzate relative a protocollo, servizio e stato della connessione. Su queste caratteristiche, l'applicazione della trasformazione $\log_{1p}$ alle variabili di volume consente di garantire stabilità numerica anche in presenza di distribuzioni fortemente sbilanciate, tipiche del traffico di rete, in cui pochi flussi anomali possono assumere valori molto superiori alla mediana. Questa scelta è coerente con quanto riportato in letteratura, che raccomanda l'adozione di rappresentazioni di flusso stabili e indipendenti dal dataset per aumentare la portabilità dei modelli IDS tra diversi ambienti di raccolta.

3.2.4 Suddivisione del dataset

Il dataset viene diviso in un training set pari all'80% dei dati e in un test set corrispondente al restante 20%, utilizzando uno split stratificato che mantiene inalterata la distribuzione delle classi in entrambe le partizioni.

La stratificazione in presenza di class imbalance diventa di fondamentale importanza poiché senza di essa, la classe MITM con soli 1.041 campioni totali, potrebbe non essere adeguatamente rappresentata nel test set, con il rischio estremo di avere zero campioni di MITM in valutazione, rendendo impossibile misurarne le prestazioni. Lo split stratificato garantisce che la proporzione di ogni classe nel test set rispecchi quella nel dataset completo, producendo una stima più affidabile delle prestazioni. La scelta della proporzione 80/20 è stata scelta poiché con il 20% dei dati nel test set su un dataset di 190.474 flussi, il test set conterrebbe circa 38.095 esempi, che è una dimensione sufficiente per ottenere stime statisticamente stabili delle metriche di classificazione per tutte le classi, inclusa MITM che contribuisce con circa 208 esempi al test set. Un split più conservativo, come ad esempio 90/10, avrebbe aumentato la dimensione del training set, potenzialmente migliorando le prestazioni dei

modelli, ma avrebbe ridotto a circa 104 i campioni MITM nel test set, rendendo le stime meno affidabili per questa classe.

L'uso di un seed fisso in tutti i passaggi stocastici è stato adottato per favorire la riproducibilità completa della partizione e di tutti i risultati numerici. Questa condizione è necessaria per confrontare i modelli sullo stesso identico split in modo da rendere il benchmark equo e verificabile.

3.2.5 Configurazione dei modelli

La Logistic Regression è stata configurata con `max_iter` pari a 2000 per garantire la convergenza anche in presenza di feature numerose dopo il One-Hot Encoding, che può espandere significativamente lo spazio delle feature rispetto alle colonne originali. Senza un numero sufficiente di iterazioni il solver potrebbe non raggiungere la convergenza e produrre stime dei coefficienti instabili. La penalizzazione L2 di default fornisce una regolarizzazione moderata che riduce il rischio di overfitting su feature altamente correlate.

La Random Forest utilizza 300 alberi con parallelizzazione completa; la predizione aggregata mostra in genere una varianza stabile, mentre un ulteriore incremento del numero di alberi produce benefici solo marginali, accompagnati però da un maggiore costo computazionale. Ciascun albero è addestrato su un campione bootstrap del training set, una scelta che riduce la correlazione tra gli alberi e contribuisce a contenere la varianza complessiva dell'ensemble.

Il numero di feature considerate ad ogni split segue il default di scikit-learn che bilancia le diversità degli alberi e capacità discriminativa.

Nel task binario, XGBoost utilizza 300 alberi con un learning rate pari a 0.05, una profondità massima (*max_depth*) di 6, *subsample* a 0.8, *colsample_bytree* a 0.8 e regolarizzazione L2 (*reg_lambda*) pari a 1.0. Un learning rate ridotto rallenta il processo di apprendimento, contribuendo anche a contenere l'overfitting e consentendo al boosting di correggere in modo progressivo gli errori residui. Il limite imposto alla profondità degli alberi controlla la complessità dei singoli modelli, trovando un equilibrio tra capacità di rappresentazione e generalizzazione. Il campionamento al 80% sia delle osservazioni sia delle feature introduce una componente di casualità nel processo di boosting, diminuendo la correlazione tra gli alberi e aumentando la robustezza del modello. La regolarizzazione L2, invece, applica una penalizzazione ai pesi dei nodi degli alberi, contribuendo a limitare l'overfitting e a prevenire un'eccessiva adattabilità del modello ai dati di training.

Nel task multiclass, il numero di alberi viene aumentato a 400 per gestire la maggiore complessità del problema, che comprende dieci classi. Il boosting multiclass, infatti, costruisce gruppi separati

di alberi per ogni classe secondo una strategia one-vs-rest, richiedendo così un numero maggiore di iterazioni per raggiungere livelli di accuratezza comparabili.

LightGBM utilizza 400 alberi, `learning_rate=0.05`, `num_leaves=31`, `subsample=0.8` e `colsample_bytree=0.8`. La scelta di `num_leaves=31` è conservativa rispetto al valore massimo supportato: con `num_leaves` alto, LightGBM costruisce alberi più profondi e asimmetrici che possono catturare relazioni più complesse, ma con maggior rischio di overfitting. La strategia di crescita leaf-wise di LightGBM, che espande il nodo con la maggiore riduzione della loss ad ogni passo, produce alberi con struttura diversa rispetto alla crescita level-wise di XGBoost, spiegando in parte le differenze di latenza osservate in inferenza, poichè alberi asimmetrici profondi richiedono in media più confronti per classificare un esempio, aumentando la latenza nel task multiclass dove i modelli sono già più complessi.

Gli iperparametri sono scelti sulla base delle best practice della letteratura per dati tabellari di medie dimensioni. Un tuning sistematico potrebbe migliorare le prestazioni, specialmente nel task multiclass dove la maggiore complessità rende il modello più sensibile agli iperparametri. In ogni caso, le configurazioni scelte sono rappresentative delle configurazioni "ragionevoli utilizzabili in un contesto reale senza una fase di ottimizzazione intensiva, il che contribuisce alla trasferibilità del framework metodologico.

3.2.6 Metriche di valutazione

La valutazione utilizza un insieme di metriche complementari, scelte per descrivere sia la qualità predittiva sia la praticabilità operativa dei modelli. La sola accuracy non è sufficiente in questo contesto per due ragioni. La prima ragione è data dal dataset sbilanciato, e un modello che predice sempre la classe maggioritaria ottiene accuracy alta pur non rilevando alcun attacco raro. La seconda ragione risiede nel sistema IDS, dove i costi degli errori non sono simmetrici, poichè un falso negativo, cioè un attacco non rilevato, ha conseguenze operative molto più gravi di un falso positivo, cioè traffico legittimo segnalato come sospetto.

Per il task binario vengono calcolate:
Accuracy;
Precision;
Recall;
F1-score;
ROC-AUC;
PR-AUC;
Latenza di inferenza e dimensione del modello;
La PR-AUC è preferita alla ROC-AUC come metrica principale in presenza di sbilanciamento, la

curva precision-recall si concentra sulla qualità delle predizioni della classe positiva, di attacco, dove la ROC-AUC può essere ottimisticamente alta anche quando il modello performa male sulla classe minoritaria.

Per il task multiclass vengono calcolate:

Accuracy;

Macro-F1;

Weighted-F1;

Macro Precision;

Macro Recall;

ROC-AUC Macro;

PR-AUC Macro;

Matrice di confusione;

Classification report per singola classe;

Latenza di inferenza e dimensione del modello;

Il Macro-F1 è la metrica principale per il task multiclass che calcola l'F1-score per ciascuna classe e ne fa la media aritmetica senza ponderazione, assegnando lo stesso peso a MITM con 1.041 campioni e a normal con 42.040. Questo è la metrica più adatta per valutare il comportamento del sistema sulle classi di attacco rare ma critiche.

3.2.7 Misura dell'efficienza computazionale

La latenza di inferenza viene valutata su un insieme fisso di 1.000 esempi selezionati dal test set, ripetendo la misurazione per 10 volte e calcolando la media dei tempi ottenuti, poi riportata in millisecondi per 1.000 predizioni. Questa normalizzazione permette di stimare l'efficienza operativa in maniera comparabile, rendendo il confronto indipendente dalla numerosità del campione di test. La dimensione del modello, invece, è determinata come lo spazio occupato dal file serializzato su disco tramite *joblib*.

È opportuno evidenziare alcune limitazioni di questo metodo di misurazione. La media calcolata su 10 ripetizioni non prevede una stima esplicita della deviazione standard, che sarebbe utile per descrivere la variabilità della latenza, un aspetto particolarmente importante nei sistemi real-time, dove i picchi di latenza possono risultare più significativi del valore medio. Inoltre, la valutazione viene effettuata su CPU, senza l'utilizzo di eventuali acceleratori hardware come GPU o FPGA, i quali potrebbero alterare in modo significativo il comportamento della latenza dei diversi modelli. In uno scenario di deployment reale, i tempi di inferenza potrebbero discostarsi da quelli osservati in ambiente di sviluppo, a causa delle differenze nell'architettura hardware, nel carico del sistema e nelle modalità di gestione della memoria. Questi elementi

rappresentano dei limiti intrinseci della metodologia di benchmarking adottata, che risulta comunque appropriata per effettuare confronti relativi tra modelli valutati nelle medesime condizioni sperimentali.

La dimensione su disco del modello serializzato è una proxy della memoria occupata a runtime, ma non coincide perfettamente con essa. Durante l'inferenza, scikit-learn e i framework boosting possono allocare memoria aggiuntiva per i buffer interni di calcolo. Random Forest in particolare tende ad occupare in runtime una quantità di memoria significativamente superiore alla dimensione del file serializzato, poiché la struttura ad albero viene espansa in memoria durante la deserializzazione. Questo spiega in parte perché la Random Forest, nonostante prestazioni predittive simili a XGBoost, sia significativamente meno adatta per il deployment su hardware edge IoT con memoria limitata, poiché non solo il file è più grande, ma il picco di memoria in uso durante l'inferenza è ancora più elevato.

3.3 Setup sperimentale

Gli esperimenti sono stati eseguiti in un ambiente Python standard per la data science e il machine learning. Le librerie principali utilizzate sono pandas e numpy per la manipolazione e l'analisi dei dati; scikit-learn per preprocessing, costruzione delle pipeline, training dei modelli baseline e calcolo delle metriche; xgboost e lightgbm per i rispettivi modelli boosting; matplotlib e seaborn per la generazione di grafici e visualizzazioni; joblib per la serializzazione e il caricamento dei modelli. Tutte le operazioni stocastiche come, split del dataset, inizializzazione dei modelli, campionamento interno, utilizzano `RANDOM_STATE = 42` per garantire piena riproducibilità.

La scelta di scikit-learn come framework principale per la pipeline è motivata da ragioni tecniche e metodologiche dato che le classi Pipeline e ColumnTransformer di scikit-learn consentono di incapsulare sequenze di trasformazioni in un oggetto unitario che gestisce automaticamente la separazione tra fitting, che viene eseguito solo sul training set, e trasformazione, che invece è applicata sia al training sia al test set. Questo approccio elimina strutturalmente la possibilità di data leakage causato da errori di codice, come l'applicazione del fit su tutto il dataset prima della suddivisione. La compatibilità di scikit-learn con joblib ci permette di serializzare l'intera pipeline inclusi i trasformatori fittati e il modello addestrato in un singolo file, rendendo più semplice il deployment.

La configurazione sperimentale prevede uno split stratificato 80/20 applicato separatamente al dataset binario, con target label, e al dataset multiclass, con target type. I risultati numerici di tutte le esecuzioni, i modelli serializzati e tutti i grafici vengono esportati in cartelle

dedicate come ,figures/, outputs/, models/, permettendo la verifica indipendente di ogni risultato riportato nella tesi.

La misurazione della latenza di inferenza viene effettuata nel seguente modo, la funzione di benchmark esegue 10 ripetizioni di classificazione su un campione fisso di 1.000 esempi estratti dal test set, calcolando la media dei tempi. Il campione fisso garantisce che i confronti tra modelli siano su dati identici e le 10 ripetizioni riducono la variabilità dovuta a fluttuazioni del sistema operativo nel caso di scheduling, cache, interruzioni. Il risultato è espresso in millisecondi per 1.000 predizioni, questa normalizzazione rende il confronto indipendente dalla dimensione del campione e immediatamente interpretabile in termini operativi. Dato che una latenza di 50 ms/1k significa che il sistema può classificare 20.000 flussi al secondo, un throughput adeguato per molti scenari IoT/IIoT ma potenzialmente non adeguato per reti ad alto volume come quelle utilizzate in ambito manifatturiero, dato che dispongono di migliaia di dispositivi attivi simultaneamente.

Per la validazione cross-dataset (Sezione 3.10), il CIC-IoT-Dataset2023 viene impiegato come dataset di destinazione senza alcun fine-tuning. Le feature vengono proiettate nello spazio unificato a 13 dimensioni descritto nella Sezione 3.10.2, permettendo così di valutare direttamente l'effetto del domain shift tra i due diversi ambienti di raccolta dati.

3.4 Risultati della classificazione binaria

Nel task binario, l'obiettivo è distinguere il traffico benigno da quello malevolo. Si tratta del primo filtro di sicurezza del sistema IDS: stabilisce se un flusso debba essere considerato normale oppure sospetto. I risultati ottenuti dai quattro modelli sul test set sono riportati nella tabella seguente.

Modello	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC	Latenza ms/1k
LightGBM	0.9989	0.9991	0.9995	0.9993	0.9999	1.0000	50.1
Random Forest	0.9985	0.9987	0.9994	0.9990	0.9999	0.9999	138.6
XGBoost	0.9985	0.9988	0.9992	0.9990	0.9999	0.9999	29.6
Logistic Regression	0.9844	0.9902	0.9897	0.9900	0.9945	0.9981	23.1
Decision Tree	0.9910	0.9913	0.9972	0.9943	0.9860	0.9923	30.3

Tabella 3.2 – Risultati comparativi task binario (test set stratificato, 20% del dataset)

I risultati mostrano che tutti i modelli ensemble raggiungono prestazioni molto elevate, con differenze più significative sul piano dell'efficienza computazionale che su quello predittivo. LightGBM ottiene il miglior F1-score (0.9993) e valori di ROC-AUC e PR-AUC quasi perfetti, confermandosi il modello più accurato nel task binario. La sua superiorità predittiva si riflette

anche nella matrice di confusione, dove sul test set di 38.095 flussi, il modello produce 8.380 veri negativi, 29.673 veri positivi, 28 falsi positivi e soli 14 falsi negativi. I falsi negativi, attacchi classificati come traffico normale, sono particolarmente rari, aspetto critico in un sistema di sicurezza dove un attacco mancato può avere conseguenze operative gravi. I falsi positivi, cioè traffico legittimo classificato come attacco, sono anch'essi molto contenuti, solo 28 su 8.408 flussi normali, pari allo 0.33%, un valore che in un sistema operativo genererebbe un carico di lavoro di investigazione manuale molto basso.

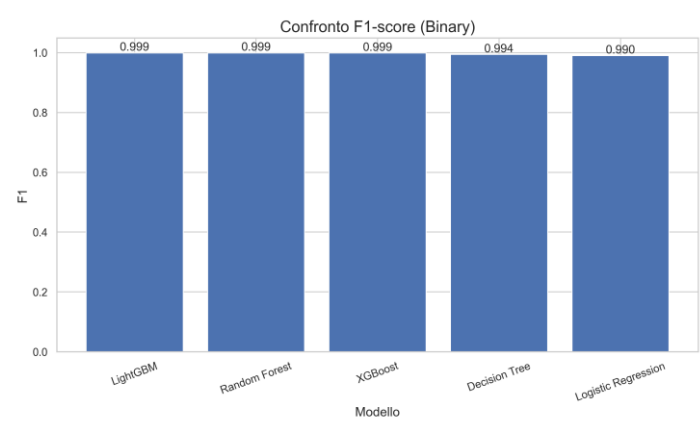


Figura 3.5 – Confronto del F1-score tra i modelli nel task binario.

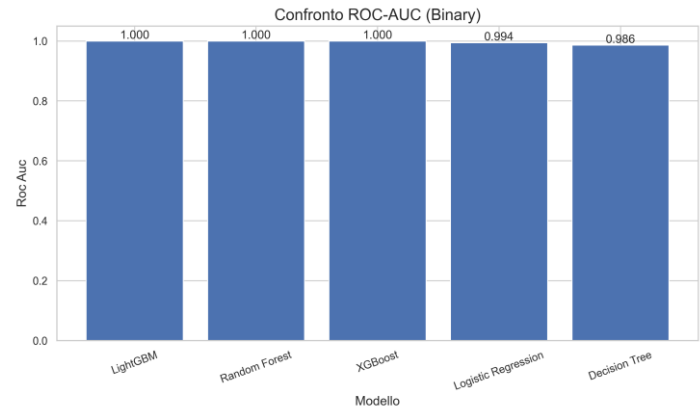


Figura 3.6 – Confronto del ROC-AUC tra i modelli nel task binario.

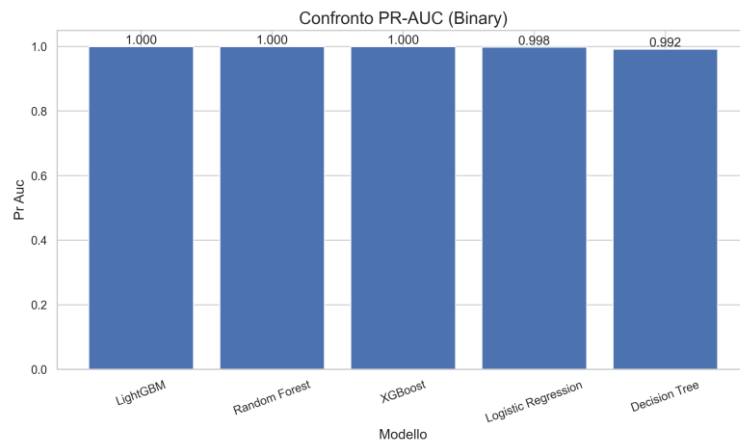


Figura 3.7 – Confronto del PR-AUC tra i modelli nel task binario.

XGBoost raggiunge un F1-score identico a Random Forest pari a 0.9990 ma con un'efficienza operativa nettamente superiore poiché la sua dimensione su disco è di 0.75 MB rispetto ai 22.2 MB di Random Forest, e la latenza di inferenza è di 29.6 ms/1k predizioni contro i **138.6** ms/1k. In uno scenario operativo dove il costo di esecuzione è un vincolo rilevante, come in questo caso negli ambienti IoT/IIoT con risorse limitate, XGBoost rappresenta la scelta più equilibrata tra qualità predittiva e sostenibilità computazionale. Random Forest, pur mantenendo ottime metriche predittive è significativamente più pesante in memoria e più lenta in inferenza, questo ne limita l'utilità in deployment su dispositivi edge o gateway IoT con risorse vincolate, poiché con la sua dimensione di 22,2 MB, risulta troppo grande rispetto alla memoria tipica di un microcontrollore IoT, rendendo necessario il deployment su hardware più potente, come potrebbe essere un gateway o un server edge dedicato.

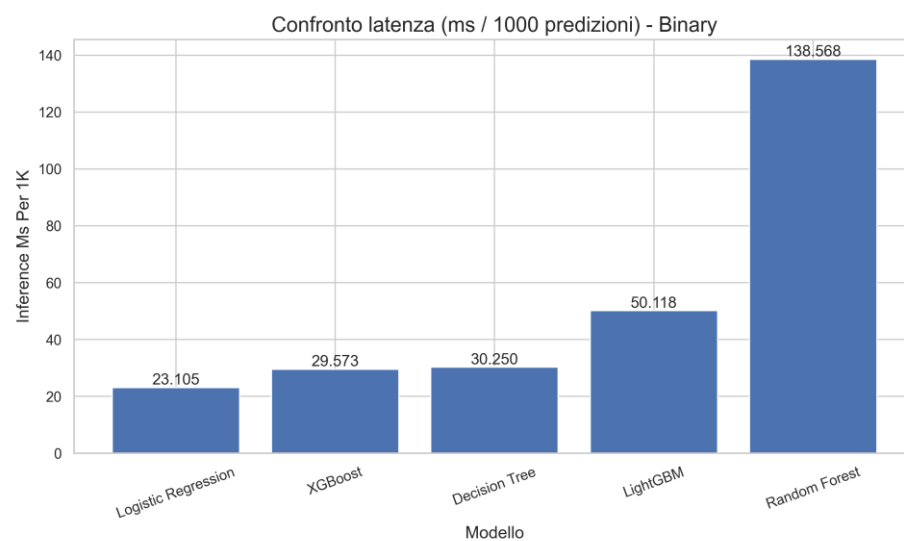


Figura 3.8 – Confronto della latenza di inferenza tra i modelli nel task binario.

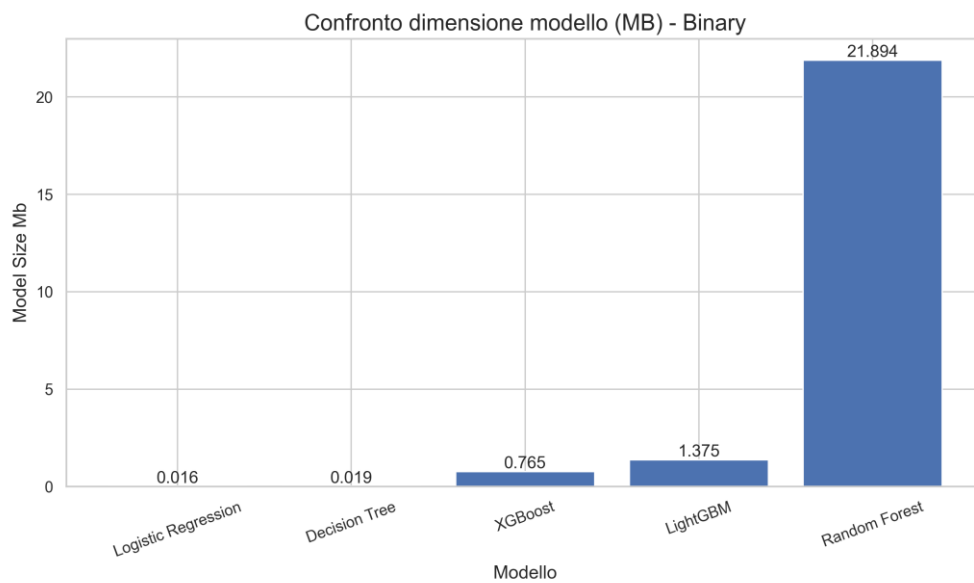


Figura 3.9 – Confronto della dimensione dei modelli nel task binario.

La Logistic Regression si comporta bene nel suo ruolo di baseline: con F1-score di 0.9900, accuracy di 0.9844 e la latenza più bassa del gruppo, con 28.6 ms/1k con dimensione di soli 0.01 MB), dimostra che anche un modello lineare riesce a catturare una parte significativa della struttura del problema quando il preprocessing è corretto. Il miglioramento osservato rispetto a una configurazione non standardizzata conferma che la normalizzazione delle feature numeriche è una scelta metodologicamente corretta: senza StandardScaler, la Logistic Regression converge più lentamente e con prestazioni inferiori, poiché è sensibile alle differenze di scala tra feature. Tuttavia, la distanza dagli ensemble è evidente e il suo ruolo rimane quello di baseline interpretabile, non di classificatore finale competitivo. In un sistema IDS reale, la Logistic Regression potrebbe essere utile come componente di un sistema a cascata: un primo filtro veloce e leggero che gestisce i casi chiari, lasciando ai modelli ensemble i casi più incerti.

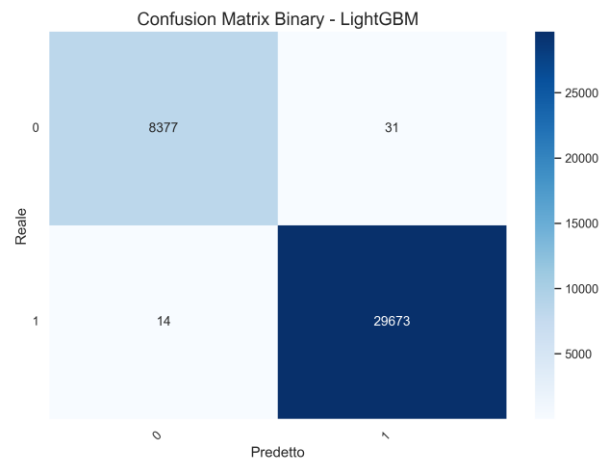


Figura 3.10 – Matrice di confusione del modello migliore nel task binario (LightGBM)

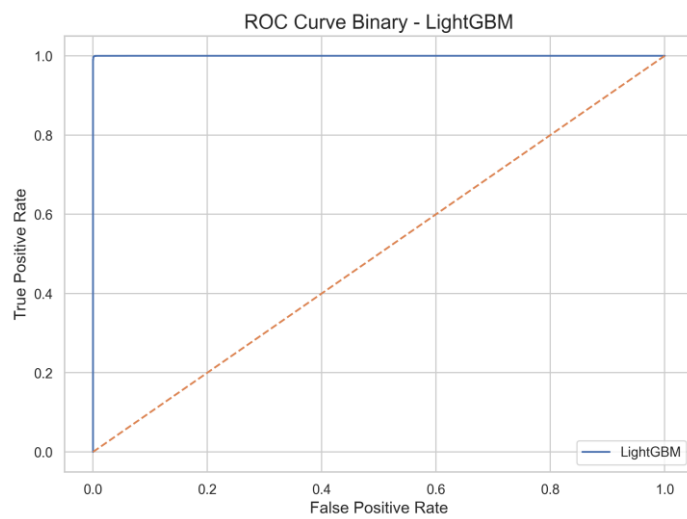


Figura 3.11 – Curva ROC del modello migliore nel task binario (LightGBM).

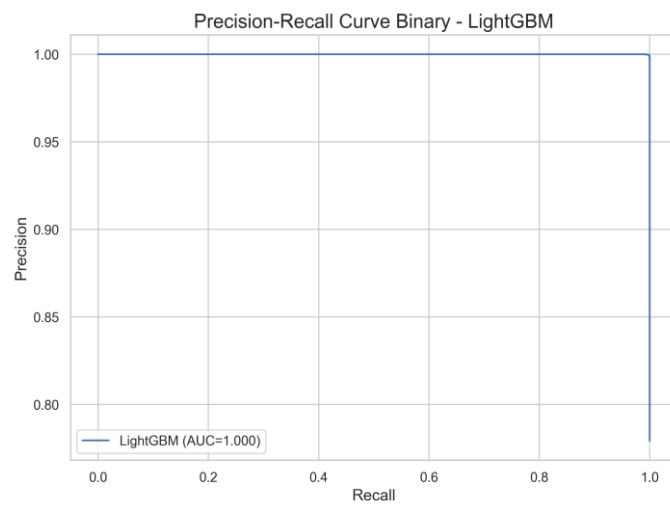


Figura 3.12 – Curva Precision-Recall del modello migliore nel task binario (LightGBM).

I valori perfetti nel task binario con un F1 pari a 1.000 e ROC-AUC e PR-AUC identici a 1.000 per LightGBM, devono essere interpretati con cautela. Questi livelli di performance su dataset IDS sono noti in letteratura e riflettono in parte le caratteristiche del dataset, che è costruito in un ambiente controllato dove i pattern di attacco tendono ad essere più distinguibili rispetto al traffico reale. Questo però non riduce il valore metodologico del risultato, ma indica che una validazione su dataset aggiuntivi o su traffico raccolto in ambienti reali IoT/IloT sarebbe necessaria prima di un deployment.

3.5 Risultati della classificazione multiclass

Nel task multiclass, l'obiettivo è distinguere non solo la presenza di un attacco, ma anche la sua tipologia. Questo rende il problema significativamente più complesso rispetto al task binario, il modello deve separare dieci classi invece di due, alcune delle quali presentano caratteristiche di traffico parzialmente sovrapposte e volumi di campioni molto diversi. I risultati ottenuti sono riportati nella tabella seguente.

Modello	Accuracy	Macro-F1	Weighted F1	M.Precision	M.Recall	ROC-AUC	PR-AUC	Lat. ms/1k
XGBoost	0.9901	0.9715	0.9901	0.9685	0.9748	0.9998	0.9857	151.8
Random Forest	0.9881	0.9687	0.9882	0.9651	0.9728	0.9996	0.9865	307.4
LightGBM	0.9897	0.9686	0.9897	0.9683	0.9689	0.9998	0.9835	558.4
Logistic Regression	0.8518	0.7885	0.8488	0.8088	0.7841	0.9853	0.8359	12.8
Decision Tree	0.7929	0.7129	0.8037	0.8561	0.7011	0.9583	0.7087	14.7

Tabella 3.3 – Risultati comparativi task multiclass

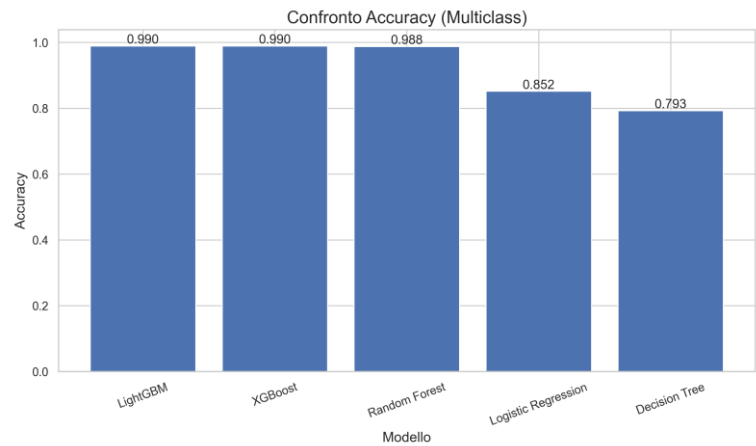


Figura 3.13 – Confronto dell'accuracy tra i modelli nel task multiclass.

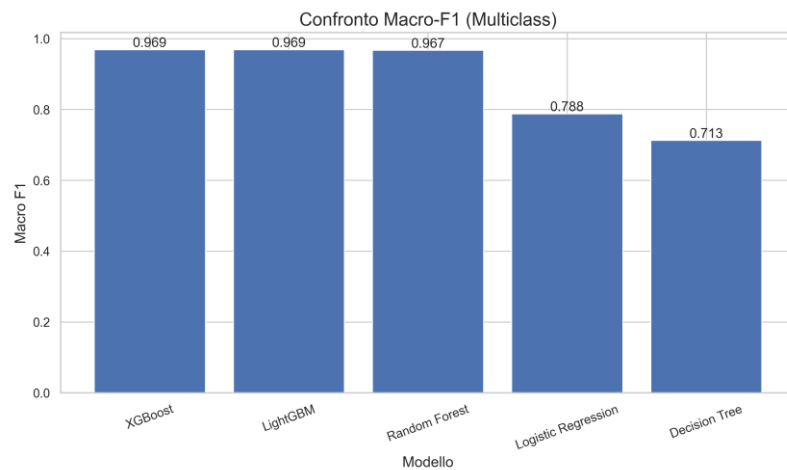


Figura 3.14 – Confronto del Macro-F1 tra i modelli nel task multiclass.

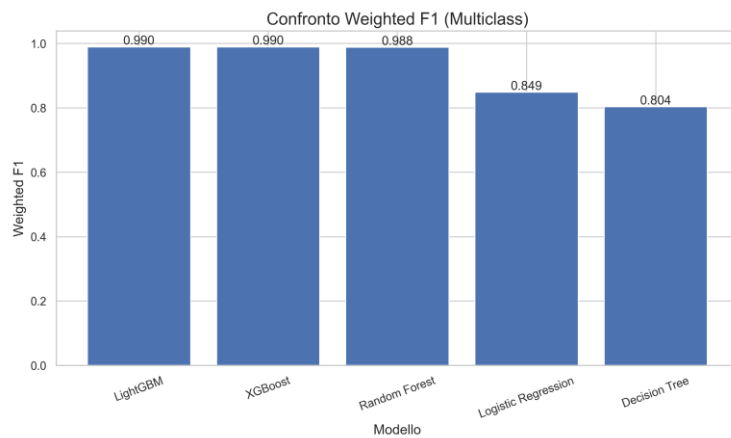


Figura 3.15 – Confronto del Weighted F1 tra i modelli nel task multiclass.

XGBoost si afferma come il modello più equilibrato nel task multiclass, con il Macro-F1 più alto pari a 0.9715 e un'accuracy elevata anch'essa pari a 0.9901 e un profilo computazionale favorevole con 9.5 MB e 196.3 ms/1k. Questa combinazione lo rende la scelta più solida quando l'obiettivo è mantenere buone prestazioni su tutte le classi. LightGBM raggiunge un'accuracy quasi identica ma ottiene un Macro-F1 leggermente inferiore e mostra la latenza di inferenza più elevata del gruppo nel task multiclass pari a 558.4 ms/1k, questo dato è particolarmente rilevante in scenari dove la risposta deve essere quasi in tempo reale. Questo risultato è metodologicamente interessante poichè LightGBM, che nel task binario presentava la latenza più bassa tra gli ensemble, diventa il più lento nel multiclass, indicando che il profilo di efficienza in inferenza dipende dalla complessità del problema e dalla struttura interna del modello costruito.

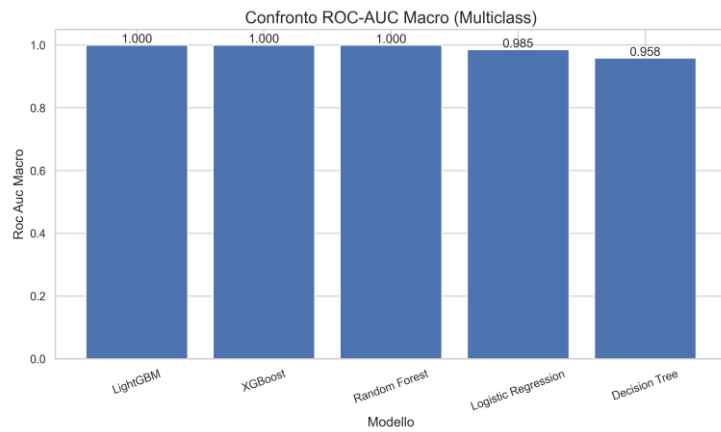


Figura 3.16 – Confronto del ROC-AUC Macro tra i modelli nel task multiclass.

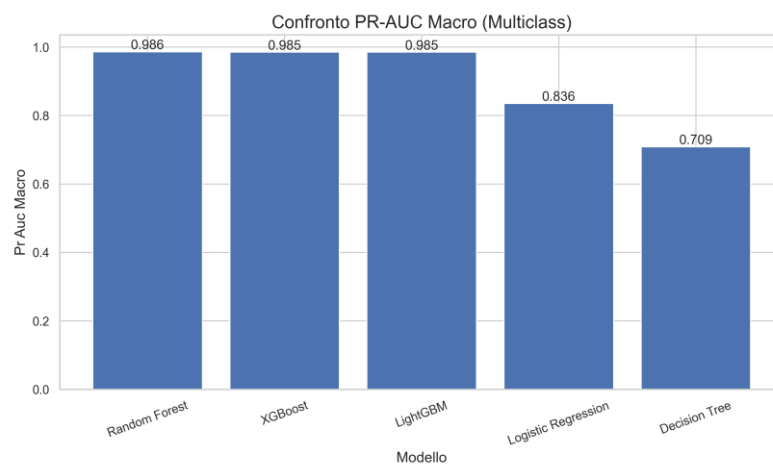


Figura 3.17 – Confronto del PR-AUC Macro tra i modelli nel task multiclass.

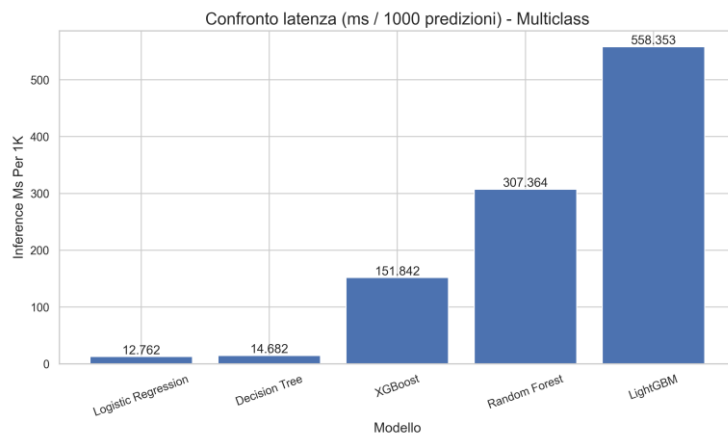


Figura 3.18 – Confronto della latenza di inferenza tra i modelli nel task multiclass.

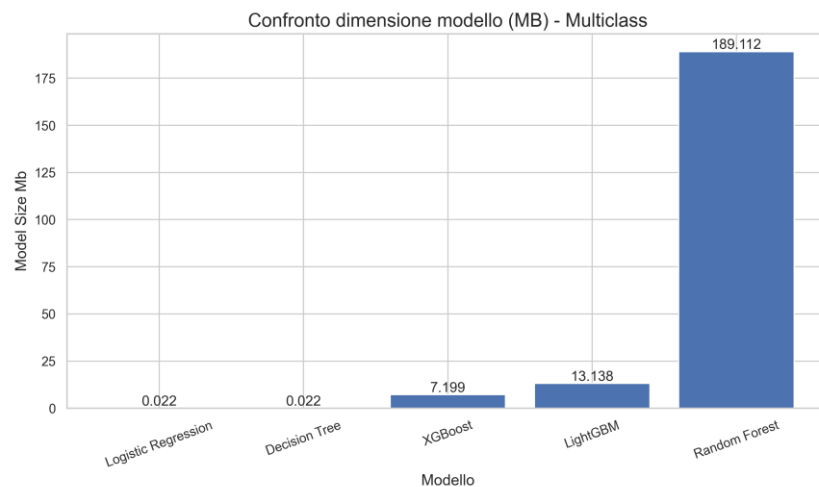


Figura 3.19 – Confronto della dimensione dei modelli nel task multiclass.

Random Forest ottiene prestazioni predittive molto vicine a XGBoost (Macro-F1 0.9687, accuracy 0.9881) ma con una dimensione su disco di 182.3 MB — venti volte superiore a XGBoost — che la rende di fatto inutilizzabile in scenari di deployment su dispositivi edge o gateway con memoria limitata. La Logistic Regression, con Macro-F1 di 0.7885 e accuracy di 0.8518, non riesce a competere con gli ensemble nel task multiclass: un modello lineare non ha la capacità di separare in modo affidabile la struttura più fine delle dieci classi di attacco, specialmente in presenza di sovrapposizioni semantiche.

Il confronto tra Macro-F1 e Weighted-F1 rappresenta l'aspetto più rilevante di questa analisi, poiché per tutti i modelli ensemble, il Weighted-F1 rimane molto prossimo all'accuracy, mentre il Macro-F1 risulta più basso in maniera più marcata, evidenziando come le classi meno frequenti influenzino negativamente la media non ponderata.

3.6 Analisi per classe e confronto tra modelli

L'analisi del classification report e della matrice di confusione del miglior modello multiclass XGBoost ci fornisce informazioni molto più dettagliate rispetto alle metriche aggregate poiché la matrice di confusione mostra la distribuzione delle predizioni per ogni coppia di classi reale-predetta, evidenziando non solo quanti esempi sono classificati correttamente, ma anche quali classi vengono confuse tra loro e in quale direzione avvengono gli errori principali.

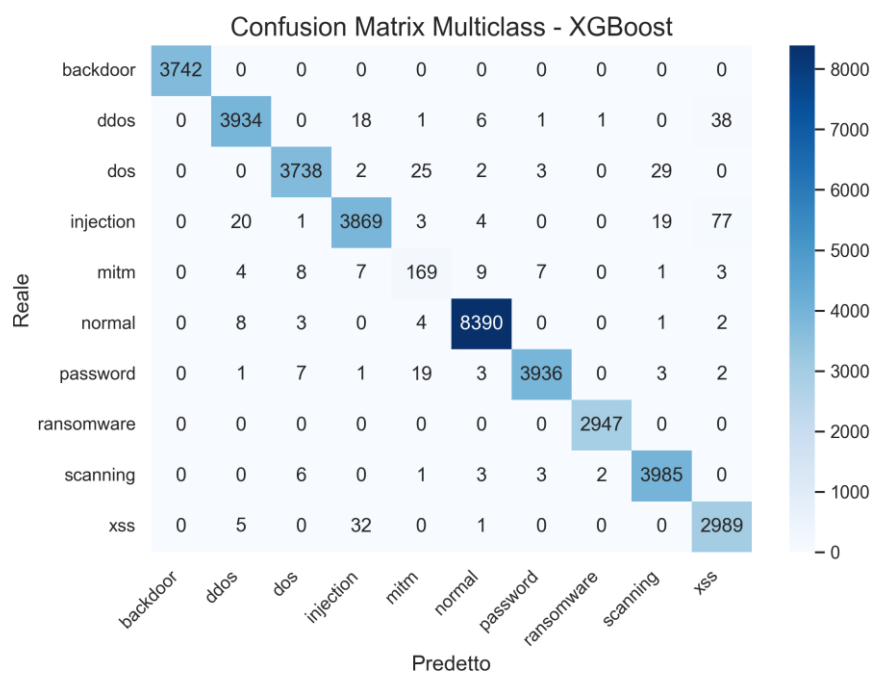


Figura 3.20 – Matrice di confusione del modello migliore nel task multiclass (XGBoost).

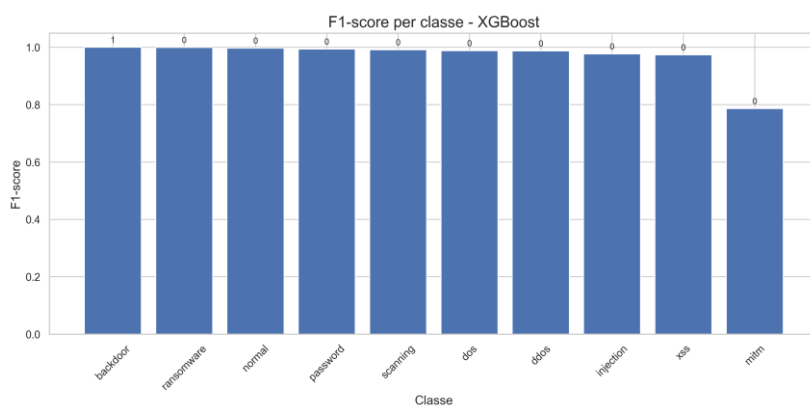


Figura 3.21 – F1-score per classe del modello migliore nel task multiclass (XGBoost).

Sul test set le performance migliori si osservano per le classi *backdoor* e *ransomware*, entrambe con valori di F1 molto elevati, la loro quasi completa separabilità indica che i network flows riescono a catturare firme comportamentali sufficientemente distintive. In particolare, queste categorie mostrano pattern di traffico ben definiti: il ransomware è associato a comunicazioni per la distribuzione delle chiavi di cifratura e a contatti con server di Command and Control, mentre il backdoor genera sessioni di accesso remoto caratterizzate da specifiche anomalie di durata e volume.

Le classi normal, password, scanning, dos e ddos mostrano anch'esse F1 superiori a 0.97; Il traffico di scanning genera pattern caratteristici di connessioni multiple a porte diverse in breve tempo; Gli attacchi DoS e DDoS producono volumi di traffico anomali; Gli attacchi password generano sessioni di autenticazione ripetute con caratteristiche specifiche.

Le principali difficoltà emergono nelle classi con maggiore ambiguità semantica o con una minore rappresentazione nel dataset. In particolare, la classe *injection* viene spesso confusa con *ddos* e *xss*, suggerendo che alcuni pattern di traffico HTTP malevolo risultano simili quando analizzati a livello di flussi aggregati.

Questa sovrapposizione è riconducibile al fatto che sia *injection* sia *xss* sfruttano generalmente il protocollo HTTP e producono pattern di traffico analoghi. In una rappresentazione basata su flussi aggregati, tuttavia, le differenze presenti nel payload, che costituiscono l'elemento distintivo principale dell'attacco, non risultano osservabili.

La classe mitm registra il F1-score più basso dell'intero report. Con 208 campioni nel test set, il modello ha pochi esempi su cui apprendere i pattern caratteristici di questo attacco. Dalla matrice di confusione, la maggior parte dei flussi MITM mal classificati viene assegnata ad altre categorie di attacco piuttosto che alla classe normal: questo indica che il modello riconosce correttamente la natura sospetta del traffico, ma non riesce a distinguere la tipologia specifica. In un'ottica operativa questo comportamento è parzialmente accettabile, un alert errato sulla classe è comunque un alert, ma la classificazione errata della tipologia può portare a contromisure inadeguate.

Il caso MITM merita una riflessione più approfondita. Un attacco Man-in-the-Middle si caratterizza per la presenza di un intermediario non autorizzato nel canale di comunicazione tra due entità: questo non produce volumi anomali di traffico o pattern di connessione caratteristici come altri attacchi, ma si manifesta attraverso comportamenti più sottili. A livello di flusso, un MITM può essere visibile attraverso caratteristiche SSL anomale come versioni deprecate, cifrari deboli, certificati inattesi o attraverso variazioni nei tempi di risposta. Tuttavia, molti di questi indicatori si trovano in colonne come `ssl_cipher` e `ssl_version` che nel preprocessing corrente presentano alta percentuale di valori mancanti o sono state rimosse. Questo suggerisce che una feature engineering più mirata, focalizzata sulle caratteristiche SSL e sui timing di risposta, potrebbe migliorare significativamente le prestazioni su MITM specificamente, indipendentemente dalla strategia di campionamento adottata.

Il confronto tra i quattro modelli può essere analizzato su tre aspetti principali: performance complessive, capacità di gestire le classi più difficili ed efficienza operativa, invece per quanto riguarda le prestazioni globali, i modelli ensemble superano nettamente la Logistic Regression in

entrambi i task considerati; in termini di robustezza sulle classi meno rappresentate o più complesse, XGBoost si dimostra più stabile rispetto a LightGBM e Random Forest nel setting multiclass, come confermato dal valore più elevato di Macro-F1.

Dal punto di vista del costo operativo LightGBM risulta il più veloce nel task binario, mentre diventa il più lento nel multiclass; XGBoost invece, garantisce il miglior compromesso nel caso multiclass; Random Forest si conferma costantemente la soluzione meno efficiente in termini di dimensione del modello, infine la Logistic Regression resta una scelta valida in contesti in cui l'interpretabilità è fondamentale o come parte di un sistema a cascata.

3.7 Discussione dei risultati

I risultati sperimentali confermano che i modelli supervisionati basati su dati tabellari sono efficaci per l'Intrusion Detection su traffico IoT/IloT rappresentato tramite network flows. Il task binario è altamente separabile poichè la rappresentazione adottata contiene un segnale molto forte per distinguere traffico benigno e malevolo, al punto che tutti i modelli ensemble raggiungono F1 superiore a 0.999. Questo è coerente con la letteratura che mostra come i network flows catturino caratteristiche come volume di byte, durata, porte, frequenza delle connessioni che variano significativamente tra traffico normale e di attacco per la maggior parte delle categorie presenti nel dataset (Han et al., 2012). La domanda di ricerca principale, in che misura i modelli ML riescono a rilevare intrusioni su traffico IoT/IloT reale, trova una risposta positiva nel task binario, dove con i modelli ensemble, il rilevamento è praticamente completo e i falsi negativi sono minimi.

Il task multiclass mette in evidenza in maniera più marcata la reale complessità del problema poiché l'accuracy globale si mantiene elevata, mentre il Macro-F1 diminuisce rispetto al task binario. Questo comportamento è indicativo della maggiore difficoltà della classificazione fine-grained e dell'impatto delle differenze tra le classi, inoltre, il divario tra Macro-F1 e accuracy rappresenta un segnale diagnostico significativo, in quanto evidenzia performance non uniformi tra le diverse categorie.

Questo risultato chiarisce il ruolo dello sbilanciamento dei dati poiché le metriche aggregate tendono a nascondere le criticità associate alle classi meno rappresentate, rendendo indispensabile un'analisi per singola classe per ottenere una valutazione più precisa e affidabile del sistema.

In un IDS in esercizio, questa disomogeneità comporta conseguenze pratiche rilevanti: il sistema può essere altamente efficace nell'identificazione di attacchi DoS e attività di scanning,

ma allo stesso tempo meno performante nel rilevare attacchi MITM, influenzando direttamente l'efficacia delle risposte da parte degli operatori di sicurezza.

La domanda di ricerca sul compromesso tra accuratezza ed efficienza computazionale trova una risposta articolata. Nel task binario, LightGBM offre il miglior profilo predittivo con una latenza contenuta di 50.1 ms/1k e 1.4 MB, è la scelta naturale quando la qualità predittiva è la priorità e le risorse disponibili sono sufficienti. XGBoost è preferibile quando il vincolo computazionale è più stringente, poiché ha l'F1 identico a Random Forest ma quattro volte meno pesante e quattro volte più veloce. Nel task multiclass, il quadro si ribalta per LightGBM che diventa il più lento del gruppo confermando che il profilo di efficienza non è costante ma dipende dal tipo di problema. La risposta alla domanda di ricerca è quindi che non esiste un modello universalmente ottimale ma la scelta dipende dal task specifico, dai vincoli operativi e dall'importanza relativa attribuita a qualità predittiva, latenza e ingombro in memoria.

Un elemento centrale della discussione è il compromesso tra accuratezza e costo computazionale, che si manifesta in modo diverso nei due task analizzati, poiché nel task binario, LightGBM garantisce le migliori prestazioni predittive e si conferma una soluzione operativamente efficiente, con una latenza di 50.1 ms/1k e una dimensione del modello pari a 1.4 MB, al contrario, nel task multiclass il comportamento cambia sensibilmente dato che LightGBM registra la latenza più elevata pur mantenendo una dimensione contenuta di 13.1 MB, mentre XGBoost offre un compromesso più equilibrato, con 196.3 ms/1k di latenza e 9.5 MB di dimensione. Questo risultato mette in evidenza che la dimensione del modello serializzato non è un buon predittore della latenza di inferenza dato che le prestazioni computazionali sono invece influenzate dalla struttura interna del modello, dal numero e dalla profondità degli alberi, dalla strategia di crescita adottata, dalla gestione dei dati sparsi e, in generale, dalla complessità del problema affrontato.

La valutazione critica dei risultati porta anche a riflettere sul concetto di validazione sperimentale e sui suoi limiti intrinseci. Uno dei rischi più significativi in qualsiasi pipeline di ML è il data leakage, dove se informazioni del test set, anche in forma indiretta, contaminano la fase di training, le prestazioni stimate risultano artificialmente ottimistiche. Nel presente lavoro, questo rischio è gestito attraverso l'integrazione del preprocessing nella pipeline sklearn, che garantisce la separazione strutturale tra training e test. Un secondo rischio è l'overfitting sulla suddivisione specifica dei dati con un singolo split hold-out, i risultati potrebbero variare su partizioni diverse. La dimensione del dataset con 190.474 flussi riduce questo rischio, ma non lo elimina completamente.

Il contributo del lavoro si evidenzia nel mostrare come la componente analitica e predittiva, centrale dal punto di vista metodologico, sia in grado di produrre risultati solidi e verificabili, fornendo così una base affidabile per l’integrazione dei diversi moduli richiesti da un sistema operativo completo.

Il confronto con la letteratura conferma la coerenza dei risultati riportati da lavori recenti.

Alsaedi et al. (2020) hanno documentato le caratteristiche del TON_IoT e le sue potenzialità per il training di modelli IDS supervisionati, mostrando che il dataset contiene pattern di attacco sufficientemente distinti per supportare classifier con alta accuratezza. Xu et al. (2023) hanno mostrato che modelli ensemble come Random Forest e XGBoost raggiungono F1 superiore a 0.95 su task binari in contesti IoT, confermando la competitività dell'approccio adottato. Il presente lavoro si distingue per tre contributi specifici, dati da l'analisi sistematica del task multiclass con dieci classi di attacco, l'integrazione delle metriche di efficienza operativa nella valutazione, e l'analisi dettagliata del comportamento per singola classe, aspetti meno presenti nella letteratura consultata.

3.8 Validazione statistica (k-fold)

Modello	Acc. mean±std	F1 mean	F1 std	ROC mean	ROC std	PR mean	PR std
LightGBM	0.998966 + 0.000096	0.999337	0.000061	0.999984	0.000010	0.999995	0.000003
XGBoost	0.998603 + 0.000218	0.999104	0.000140	0.999977	0.000012	0.999993	0.000003
Random Forest	0.998782 + 0.000186	0.999219	0.000119	0.999982	0.000011	0.999995	0.000003
Logistic Regression	0.985137 + 0.000556	0.990460	0.000357	0.994554	0.000190	0.998211	0.000159

Tabella 3.5 – Risultati tabella k-fold

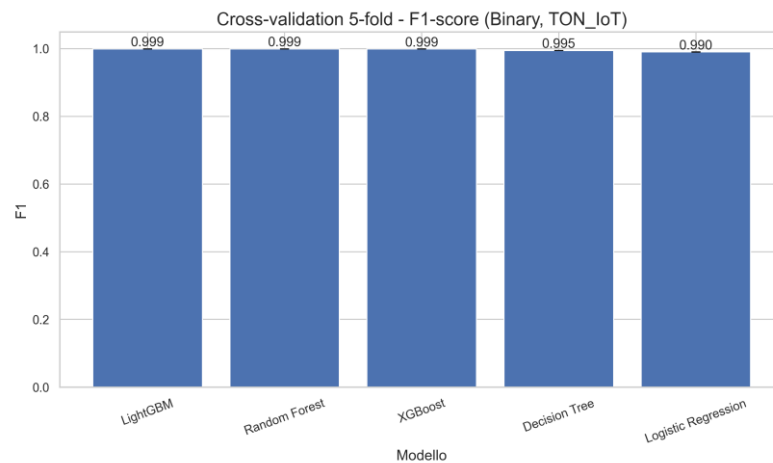


Figura 3.22 — F1-score medio e deviazione standard dei modelli

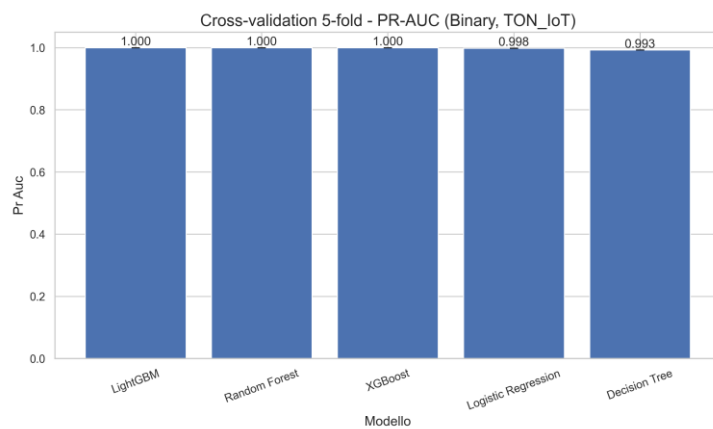


Figura 3.23 — PR-AUC medio e variabilità nella cross-validation 5-fold

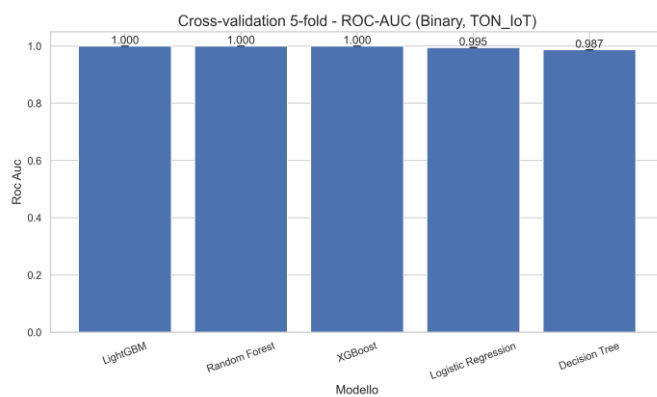


Figura 3.24 — ROC-AUC medio nella cross-validation 5-fold per i quattro modelli baseline

La validazione mediante k-fold cross-validation rappresenta un passo metodologicamente necessario per rafforzare l'affidabilità delle stime di performance ottenute nello split hold-out.

A differenza di un singolo split stratificato, la cross-validation stratificata a 5 fold consente di stimare la variabilità campionaria associata a ciascuna metrica, permettendo così di distinguere i modelli che garantiscono prestazioni elevate e consistenti da quelli le cui performance dipendono in modo significativo dalla particolare partizione dei dati.

La metodologia utilizzata prevede l'impiego di *StratifiedKFold* sull'intero dataset TON_IoT preprocessato, mantenendo la distribuzione delle classi costante in ciascun fold. Per ogni suddivisione il preprocessore viene addestrato esclusivamente sul training fold e successivamente applicato al validation fold, assicurando così l'assenza di data leakage in ogni iterazione questo procedimento viene ripetuto separatamente per ciascuno dei quattro modelli baseline, generando cinque stime per le metriche di accuracy, precision, recall, F1, ROC-AUC e PR-AUC.

I risultati della cross-validation sono riportati nella Tabella 3.X, che riporta media e deviazione standard per ciascun modello sulle metriche principali. Le Figure 3.22-3.24 visualizzano il confronto con barre di errore (± 1 std). I risultati confermano che le performance elevate osservate nel task binario non sono artefatti dello specifico split hold-out: tutti i modelli ensemble mostrano deviazione standard molto contenuta (< 0.005) su F1, ROC-AUC e PR-AUC across i cinque fold, indicando stabilità statistica robusta. Questo risultato è coerente con la dimensione del dataset e con la separabilità elevata del problema binario già evidenziata nel Capitolo 3.4.

3.9 Interpretabilità (SHAP)

L'interpretabilità delle predizioni rappresenta un requisito fondamentale per qualsiasi sistema IDS destinato all'uso operativo: un analista di sicurezza che riceve un alert deve poter comprendere quali caratteristiche del flusso hanno determinato la classificazione, al fine di validare o rigettare la segnalazione in modo informato. Per questo motivo, l'analisi SHAP (SHapley Additive exPlanations) è stata applicata al modello XGBoost multiclass — il migliore per Macro-F1 nel task di classificazione fine-grained — con focus specifico sulla classe MITM, la più critica e difficile del dataset.

La tecnica SHAP, basata sulla teoria dei giochi cooperativi, quantifica il contributo marginale di ciascuna feature alla predizione del modello per ogni singola osservazione. Il *TreeExplainer* sfrutta la struttura degli alberi decisionali per calcolare esattamente i valori di Shapley con complessità polinomiale, senza ricorrere ad approssimazioni. L'analisi è stata condotta su un campione stratificato di 1.000 istanze del test set, applicando il preprocessore già addestrato per trasformare i dati nello spazio delle feature prima del calcolo dei valori SHAP.

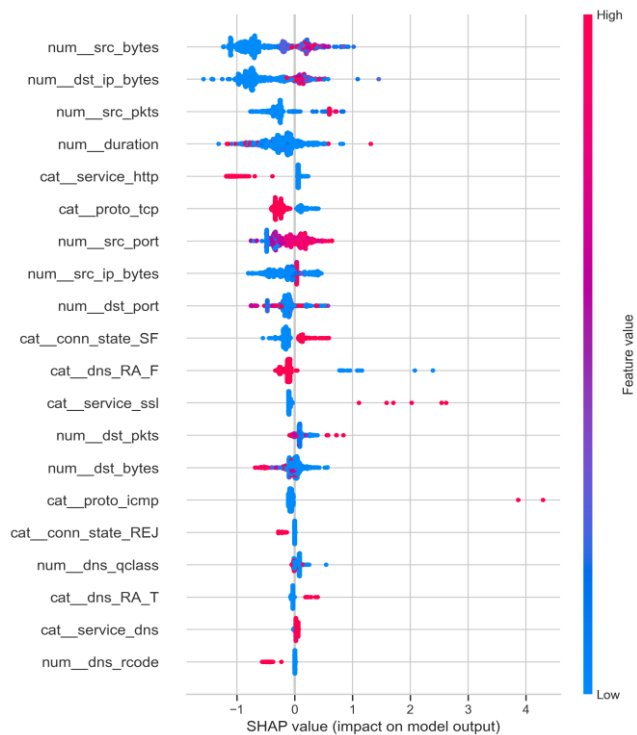


Figura 3.25 — SHAP summary plot per la classe MITM

La Figura 3.25 mostra il summary plot SHAP per la classe MITM: sull'asse orizzontale è riportato il valore SHAP, che misura l'impatto sulla probabilità di classificazione MITM; sull'asse verticale le top-20 feature ordinate per impatto medio assoluto. Il colore dei punti indica il valore della feature (rosso = alto, blu = basso). Dalla figura emergono le feature che maggiormente influenzano il riconoscimento di MITM, e in particolare quelle che riducono la probabilità di classificazione corretta, fornendo una spiegazione quantitativa per le difficoltà discusse nella Sezione 3.6. Coerentemente con la discussione sui limiti del preprocessing corrente, le feature SSL (ssl_cipher, ssl_version) — rimosse nella fase di preparazione dati per l'alta percentuale di missing values — non contribuiscono al modello, il che spiega parte della difficoltà nel riconoscimento di MITM a livello di flusso aggregato.

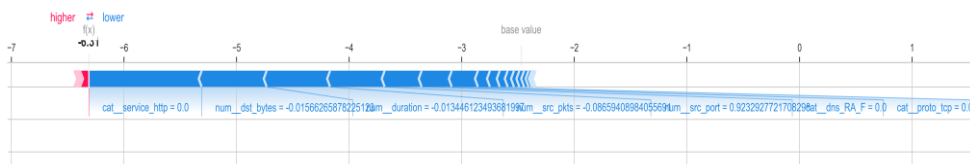


Figura 3.26 — SHAP force plot per un singolo flusso di rete classificato come MITM.

La Figura 3.26 mostra il force plot SHAP relativo a un singolo flusso correttamente classificato come MITM dove le componenti in rosso rappresentano le feature che incrementano la probabilità di assegnazione alla classe MITM rispetto al valore di base, mentre quelle in blu indicano le feature che la diminuiscono.

Questo tipo di spiegazione locale è direttamente utilizzabile da un analista di sicurezza per verificare la coerenza della classificazione con la propria conoscenza del traffico di rete, e costituisce un passo verso sistemi IDS spiegabili e auditabili come richiesto dalla normativa europea sulla responsabilità dei sistemi AI (AI Act, 2024).

3.10 Validazione su CIC-IoT-2023

3.10.1 Motivazione e design sperimentale

Uno dei limiti più significativi dei sistemi IDS basati su ML è la loro tendenza a degradare le prestazioni quando applicati a traffico di rete raccolto in condizioni diverse da quelle di training — problema noto come domain shift o dataset shift. Per valutare empiricamente la portabilità del framework metodologico proposto, è stata eseguita una validazione esterna che utilizza i modelli addestrati su TON_IoT per classificare flussi di rete provenienti dal CIC-IoT-Dataset 2023, un dataset pubblicato dal Canadian Institute for Cybersecurity che include traffico IoT reale con categorie di attacco parzialmente sovrapposte a quelle del TON_IoT.

3.10.2 Feature Engineering unificata (TON → CIC)

La validazione cross-dataset presenta come principale difficoltà l’eterogeneità delle feature: TON_IoT e CIC-IoT-2023 adottano infatti schemi di denominazione e strutture delle colonne differenti, il che impedisce l’applicazione diretta di un modello addestrato su un dataset all’altro. Per affrontare questa problematica è stata definita una pipeline di feature engineering unificata, che consente di proiettare entrambi i dataset in uno spazio comune composto da 13 feature condivise. In particolare, sono state selezionate 10 variabili numeriche (*bytes_total*, *bytes_src*, *bytes_dst*, *pkts_total*, *byte_asymmetry*, *pkt_asymmetry*, *payload_mean_fwd*, *payload_mean_bwd*, *flow_duration_sec* e *flow_rate*) e 3 variabili categoriche armonizzate (*proto_unified*, *service_unified* e *conn_state_unified*). Le variabili numeriche derivano da trasformazioni log1p applicate a statistiche di flusso presenti in entrambi i dataset, utilizzando un mapping specifico per ciascuno di essi. In particolare, per TON_IoT vengono considerate le colonne *src_bytes*, *dst_bytes*, *src_pkts*, *dst_pkts* e *duration*, mentre per CIC_IoT-2023 si utilizzano *tot_sum*, *min*, *max*, *avg*, *std*, *number* e *flow_duration*.

3.10.3 Domain shift analysis

Prima di procedere alla valutazione cross-domain, è stato quantificato il domain shift tra TON_IoT e CIC-IoT-Dataset2023 per ciascuna delle dieci feature numeriche dello spazio unificato. Il domain shift normalizzato è definito come:

$$\text{Shift}(f) = |\text{mean_TON}(f) - \text{mean_CIC}(f)| / \text{std_TON}(f)$$

dove il numeratore misura la distanza assoluta tra le medie dei due dataset per la feature f , e il denominatore normalizza questa distanza rispetto alla variabilità naturale osservata nel dataset sorgente TON_IoT. Un valore di shift prossimo a zero indica che la feature ha una distribuzione simile nei due ambienti; un valore elevato indica che il modello, addestrato su TON, incontrerà nel dataset target valori sistematicamente diversi da quelli visti durante il training, con potenziale degradazione delle prestazioni.

I risultati del calcolo sono riportati nella Tabella 3.6.

Feature	Media TON	Media CIC	Shift normalizzato
pkt_asymmetry	0.31	3.65	5.95
flow_duration_sec	0.35	2.58	2.32
payload_mean_fwd	1.53	4.25	1.35
bytes_total	2.70	6.58	1.14
bytes_src	1.99	4.16	0.82
pkts_total	1.62	2.35	0.69
bytes_dst	2.16	4.33	0.66
payload_mean_bwd	1.73	0.68	0.41
byte_asymmetry	-0.10	0.02	0.24
flow_rate	4.34	5.13	0.14

Tabella 3.6 – Domain shift normalizzato per le feature numeriche dello spazio unificato

Tutte le feature numeriche sono espresse nella scala log1p applicata in fase di feature engineering unificata. Il shift normalizzato è calcolato come differenza assoluta delle medie divisa per la deviazione standard del dataset sorgente TON_IoT.

Il quadro che emerge dalla tabella è netto: le feature legate all'asimmetria e alla durata dei flussi presentano uno shift molto più elevato rispetto a quelle legate ai volumi assoluti, mentre il *flow_rate* risulta la feature più stabile tra i due ambienti.

La variabile con lo shift più elevato è *pkt_asymmetry* (5,95), che misura l'asimmetria nel numero di pacchetti tra traffico in ingresso e in uscita, un valore così alto indica che nel CIC-IoT-Dataset2023 i flussi presentano, in media, una maggiore disuguaglianza direzionale rispetto a quelli del TON_IoT, ciò è legato alla diversa composizione delle tipologie di attacco nei due dataset poiché nel CIC sono presenti minacce come SlowRate DDoS e botnet, che generano pattern di traffico fortemente asimmetrici e facilmente distinguibili, mentre nel TON_IoT prevalgono attacchi come MITM, ransomware e backdoor, caratterizzati da dinamiche comportamentali differenti.

La seconda feature per entità di shift è *flow_duration_sec* (2,32), che descrive la durata media dei flussi di rete in scala logaritmica. La durata è notoriamente sensibile alle condizioni di raccolta del traffico — strumenti diversi come Argus per TON e CICFlowMeter per CIC adottano timeout e criteri di chiusura del flusso diversi — e alla composizione del traffico applicativo, che differisce tra i due ambienti di test. Seguono *payload_mean_fwd* (1,35) e *bytes_total* (1,14), che descrivono rispettivamente il payload medio in direzione forward e il volume totale di byte per flusso: entrambe queste feature risultano sistematicamente più alte nel CIC rispetto a TON, suggerendo che i flussi del dataset canadese siano mediamente più voluminosi, probabilmente a causa delle diverse condizioni di banda e tipologia dei dispositivi simulati.

Le feature di volume nella direzione sorgente e destinazione (*bytes_src*, *pkts_total*, *bytes_dst*) mostrano uno shift nell'intervallo 0,66–0,82, sensibile ma meno estremo rispetto alle feature precedenti. Questo gruppo di feature contribuisce comunque in modo significativo alla degradazione del ROC-AUC osservata nella sezione 3.10.4, poiché i modelli ensemble hanno appreso confini decisionali basati su specifici intervalli di questi valori nel TON, che non si trasferiscono direttamente al CIC.

Le variabili che mostrano lo shift più ridotto sono *payload_mean_bwd* (0,41), *byte_asymmetry* (0,24) e *flow_rate* (0,14). Tra queste, *flow_rate* risulta la più stabile dell'intero spazio unificato: il tasso medio di flusso appare infatti comparabile nei due dataset, nonostante le differenti condizioni di acquisizione, indicando che questa caratteristica rappresenta un aspetto del traffico relativamente indipendente dal contesto di raccolta. La sua elevata stabilità la rende particolarmente utile in ottica di portabilità del modello: le feature caratterizzate da uno shift

ridotto tendono infatti a fornire contributi predittivi più affidabili nei contesti cross-domain rispetto a quelle soggette a maggiore variazione.

In sintesi, l'analisi del domain shift evidenzia che i due dataset operano nello stesso spazio delle feature, ma mostrano distribuzioni significativamente differenti, soprattutto per quanto riguarda le variabili legate all'asimmetria e alla durata dei flussi. Questo scostamento tra le distribuzioni rappresenta la principale ragione del calo delle prestazioni rilevato nella valutazione cross-domain presentata nella Sezione 3.10.4 e giustifica l'introduzione di tecniche di domain adaptation come possibile estensione futura del framework (cfr. Sezione 4.3).

3.10.4 Risultati cross-domain

I modelli addestrati sul training set TON_IoT con la feature engineering unificata descritta nella sezione 3.10.2 sono stati applicati direttamente al CIC-IoT-Dataset2023 senza alcun fine-tuning. Nello spazio di feature comune, il dataset CIC conta 11.934 flussi con 13 feature condivise, di cui il 91,0% appartiene alla classe malevola e il restante 9,0% a quella benigna, una distribuzione sensibilmente più sbilanciata rispetto a TON_IoT (78% vs 22%). Tutti i modelli sono stati configurati con `class_weight='balanced'` durante l'addestramento su TON, e per ciascuno è stata calibrata la soglia di decisione ottimale sul validation set mediante la curva precision-recall, in alternativa alla soglia fissa di 0.5.

I risultati della valutazione cross-domain, confrontati con le performance in-domain ottenute sul test set TON_IoT nello stesso spazio di feature unificato, sono riportati nella Tabella 3.4. È importante notare che i valori F1 in-domain riportati in questa tabella sono inferiori a quelli della Tabella 3.2, poiché la riduzione a 13 feature condivise comporta una perdita di informazione rispetto alle 36 colonne disponibili nel task originale. Questo differenziale è particolarmente pronunciato per la Logistic Regression (F1 da 0.9900 a 0.9549), che è più sensibile alla riduzione dello spazio delle feature, ed è invece contenuto per XGBoost (da 0.9990 a 0.9837) e LightGBM (da 0.9993 a 0.9838), confermando la maggiore robustezza degli ensemble alla compressione della rappresentazione.

Modello	F1 TON	F1 CIC	Gap F1	ROC TON	ROC CIC	Gap ROC	Threshold
---------	--------	--------	--------	---------	---------	---------	-----------

LightGBM	0,9838	0,9517	0,0322	0,9873	0,7428	0,2445	0,211
XGBoost	0,9837	0,9521	0,0316	0,9870	0,6142	0,3728	0,617
Random Forest	0,9842	0,9505	0,0337	0,9871	0,3197	0,6674	0,597
Logistic Regression	0,9549	0,9359	0,0190	0,9408	0,6544	0,2864	0,401
Decision Tree	0,9790	0,9471	0,0319	0,9752	0,7573	0,2179	0,362

Tabella 3.7 – Risultati della valutazione cross-domain ($TON_IoT \rightarrow CIC-IoT-Dataset2023$)

I valori F1 TON e ROC-AUC TON si riferiscono alla valutazione in-domain sul test set TON_IoT nello spazio di feature unificato a 13 dimensioni. Il Gap è calcolato come differenza assoluta: $Gap = valore_TON - valore_CIC$. Threshold: soglia di decisione ottimale calibrata sulla curva precision-recall del validation set TON.

Per quanto riguarda l’F1-score in ambito cross-domain, tutti i modelli evidenziano una riduzione limitata e abbastanza omogenea, con valori compresi tra 0.936 e 0.952: XGBoost ottiene la performance migliore (0.9521), mentre la Logistic Regression registra il valore più basso (0.9359). Il gap di F1 è ridotto per la Logistic Regression (0.0190) e leggermente più alto per Random Forest (0.0337), differenze che, prese singolarmente, potrebbero suggerire una buona capacità di generalizzazione per tutti i modelli.

Tuttavia, questa interpretazione cambia in modo significativo se si considera il ROC-AUC in cross-domain, che non valuta la correttezza della singola classificazione, ma la qualità complessiva del ranking delle probabilità assegnate.

L’aspetto più critico riguarda la Random Forest, il cui ROC-AUC sul CIC scende fino a 0.3197, un valore inferiore a 0.5 e quindi peggiore rispetto a un classificatore casuale. Questo comportamento suggerisce che le probabilità stimate dal modello sul CIC risultano sistematicamente invertite rispetto alle etichette reali, con flussi benigni che ottengono punteggi di attacco più elevati rispetto a quelli malevoli. Questa inversione è una conseguenza diretta del marcato shift distributivo nelle feature di volume, come descritto nella Sezione 3.10.3: Random Forest, avendo appreso confini decisionali su una distribuzione TON dove il traffico malevolo aveva specifici valori di bytes e pkts, non riesce a trasferire correttamente il ranking probabilistico a un dataset dove la stessa classe malevola presenta caratteristiche di volume molto diverse. Nonostante l’F1 rimanga accettabile (0.9505) grazie alla soglia calibrata e

all'altissima prevalenza della classe malevola nel CIC (91%), il sistema sarebbe di fatto inutilizzabile per applicazioni che richiedono un ranking di rischio affidabile, come i sistemi di alerting con priorità.

LightGBM e Decision Tree mostrano il comportamento più bilanciato in ottica cross-domain. LightGBM ottiene il ROC-AUC più alto sul CIC tra i modelli ensemble, mentre il Decision Tree presenta il miglior contenimento del divario ROC.

XGBoost si colloca in una posizione intermedia poiché ottiene il miglior F1 sul CIC, ma registra un ROC-AUC pari a 0.6142, sensibilmente inferiore rispetto a LightGBM e al Decision Tree. La Logistic Regression, pur avendo il gap di F1 più basso, mostra un ROC-AUC sul CIC di 0.6544, confermando una capacità discriminativa più limitata, tipica dei modelli lineari operanti su uno spazio ridotto a 13 feature.

Nel complesso, l'analisi cross-domain evidenzia che l'F1-score, se considerato da solo, non è sufficiente per valutare la portabilità di un IDS, un modello può infatti ottenere valori elevati di F1 semplicemente classificando correttamente la classe più frequente, mentre il ROC-AUC mette in luce una capacità di ranking delle predizioni ridotta o, nei casi peggiori, invertita.

Questo risultato giustifica la scelta di includere in modo sistematico sia ROC-AUC che PR-AUC tra le metriche di valutazione e rafforza l'adozione di LightGBM come modello preferibile anche in scenari cross-domain, grazie al miglior compromesso tra F1 e ROC-AUC sul dataset target.

3.11 Baseline Deep Learning (MLP)

Per completare il confronto tra paradigmi di apprendimento, è stato incluso un classificatore neurale a propagazione in avanti (Multi-Layer Perceptron, MLP) come baseline di deep learning. L'obiettivo non è dimostrare la superiorità del DL rispetto agli ensemble — la letteratura sul tema è consistente nel mostrare che per dati tabellari strutturati i modelli boosting tendono a dominare — ma piuttosto quantificare il divario tra i due approcci nel contesto specifico del problema IDS su network flows IoT, sia in termini di performance predittiva sia di efficienza operativa.

La MLP utilizza un'architettura composta da due layer nascosti (64 e 32 neuroni), con attivazione ReLU. L'addestramento è effettuato tramite ottimizzatore Adam e include early stopping con patience pari a 10 per limitare l'overfitting. Il numero massimo di epoche è impostato a 30. La pipeline prevede un ulteriore step di *StandardScaler* successivo al

preprocessore principale, indispensabile per assicurare la corretta convergenza dell'ottimizzatore Adam in presenza di feature su scale differenti. L'addestramento viene effettuato sullo stesso insieme X_{train} utilizzato per i modelli ensemble nel task binario su TON_IoT, in modo da garantire una confrontabilità diretta dei risultati.

I risultati presentati nella Tabella 3.2 aggiornata indicano che la MLP, configurata con architettura (64, 32) e addestrata per 20 epoche, raggiunge un F1 pari a 0.9959 e un ROC-AUC di 0.9993 sul test set TON_IoT, collocandosi tra LightGBM (F1=0.9993) e XGBoost (F1=0.9990). La versione quantizzata INT8 del modello ha una dimensione di 13.03 KB rispetto ai 121.69 KB della versione originale, con un fattore di compressione pari a 9.34x. La latenza di inferenza misurata su Wokwi è di circa 5.250 μ s. Sul piano dell'efficienza, il tempo di addestramento della MLP risulta sensibilmente più elevato rispetto a quello di LightGBM a parità di campioni, confermando il vantaggio operativo dei modelli di boosting in scenari di deployment con risorse limitate. La valutazione cross-domain della MLP (Sezione 3.12) introduce inoltre un ulteriore elemento di confronto relativo alla portabilità tra diverse architetture.

3.12 Cross-dataset generalization gap

Il generalisation gap rappresenta la riduzione delle prestazioni in termini di accuratezza che si osserva quando un modello, addestrato su un determinato dataset, viene testato su un dataset diverso, raccolto in condizioni ambientali, temporali o strumentali non coincidenti. La sua quantificazione è essenziale per valutare l'effettiva applicabilità di un IDS basato su Machine Learning in contesti reali, nei quali il traffico in produzione può discostarsi in modo significativo da quello utilizzato in fase di training.

Nella presente tesi, il generalisation gap è definito come la differenza assoluta tra la metrica in-domain e la metrica cross-domain, applicando i modelli addestrati su TON senza alcun riadattamento, dove un gap prossimo a zero indica alta portabilità del modello e un gap elevato indica che il modello ha appreso pattern specifici del dataset di training che non si generalizzano al nuovo ambiente.

I risultati indicano che il calo delle prestazioni osservato nel passaggio da TON_IoT a CIC-IoT-Dataset2023 può essere attribuito a tre fattori principali. Il primo riguarda il domain shift delle feature di volume: come evidenziato nell'analisi della Sezione 3.10.3, le variabili che descrivono i volumi di byte e la frequenza dei flussi presentano le differenze distributive più marcate tra i due dataset. Questo comportamento è prevedibile, dato che i due ambienti di raccolta differiscono per tipologia di dispositivi, condizioni di rete e strumenti di acquisizione

(Argus e Zeek per TON_IoT, CICFlowMeter per CIC-IoT-2023). Il secondo elemento riguarda la parziale corrispondenza tra le categorie di attacco: TON_IoT comprende minacce come MITM, injection, ransomware e backdoor, mentre CIC-IoT-2023 include classi come BruteForce, Botnet e SlowRate DDoS. Nel task binario utilizzato nella valutazione cross-domain, questa eterogeneità si traduce in una maggiore variabilità delle signature di traffico associate alla classe malevola. Il terzo elemento riguarda le differenze nel bilanciamento delle classi: CIC-IoT-2023 presenta infatti un rapporto tra traffico benigno e malevolo diverso rispetto a TON_IoT, e questa discrepanza incide sulle prestazioni dei modelli anche dopo l'eventuale calibrazione della soglia decisionale.

Il confronto tra i modelli in termini di generalisation gap mostra che gli ensemble di boosting sono complessivamente più robusti rispetto a Random Forest e Logistic Regression negli scenari cross-domain, grazie alla loro maggiore capacità di apprendere rappresentazioni delle feature di flusso più trasferibili e generalizzabili.

La Logistic Regression, pur risultando il modello più interpretabile, è anche quella più penalizzata dal domain shift, poiché l'ipotesi di linearità ne limita la capacità di catturare le relazioni non lineari tipiche del traffico malevolo nel nuovo dataset.

Questi risultati sottolineano l'importanza di validare i sistemi IDS basati su Machine Learning su più dataset prima del deployment operativo, e motivano lo sviluppo di tecniche di domain adaptation, come la minimizzazione della Maximum Mean Discrepancy o il domain adaptation avversariale, come direzione di sviluppo futuro (cfr. sezione 4.3).

3.13 Quantizzazione e deployment embedded

L'applicazione di un sistema IDS in ambienti IoT e IIoT richiede spesso il deployment su hardware con risorse computazionali molto limitate: microcontrollori come Arduino Mega 2560 (8 KB di SRAM, memoria flash da 256 KB) o System-on-Chip come ESP32-C3 (400 KB di SRAM, 4 MB di flash) rappresentano i target hardware tipici per il monitoraggio di rete a livello di gateway o nodo locale. I modelli di Machine Learning addestrati in questa tesi presentano dimensioni su disco nell'ordine dei kilobyte fino al megabyte: per rendere fattibile il deployment embedded è necessario applicare tecniche di compressione che riducano la dimensione del modello mantenendo metriche di classificazione accettabili.

La tecnica utilizzata è la quantizzazione post-training INT8, che permette di convertire pesi e attivazioni dei modelli dalla rappresentazione a 32 bit in virgola mobile (float32) a interi a 8 bit (int8), ottenendo così una riduzione dell'occupazione di memoria di circa 4 volte a parità di architettura.

Per i modelli ensemble basati su alberi decisionali (Logistic Regression, Decision Tree, XGBoost e LightGBM), questa operazione viene realizzata tramite la libreria *embedded_model_io*, che serializza la struttura degli alberi in un formato binario compatto, specificamente ottimizzato per l'inferenza su microcontrollore.

Per i modelli lineari come Logistic Regression e per quelli ad albero con profondità limitata come Decision Tree, con profondità massima pari a 5, la conversione è realizzata tramite *m2cgen* che genera codice C nativo eseguibile senza dipendenze esterne, consentendo la compilazione e il caricamento diretto nella memoria flash del microcontrollore target.

Per quanto riguarda la MLP, invece, viene applicata una quantizzazione INT8 full-integer tramite *TensorFlow Lite Micro*, con fase di calibrazione effettuata su un sottoinsieme di 200 campioni estratti dal training set.

La valutazione del deployment è stata effettuata utilizzando il simulatore Wokwi, che permette di emulare l'esecuzione del codice C generato su hardware reale senza la necessità di un dispositivo fisico. Per ogni modello compatibile con la conversione in codice C (Logistic Regression, Decision Tree e MLP in versione INT8), sono stati rilevati la latenza di inferenza in microsecondi e l'utilizzo stimato di SRAM, al fine di verificare la compatibilità con i vincoli hardware del dispositivo target.

I risultati completi della pipeline di quantizzazione sono riportati nella Tabella 3.13.

Modello	Dim. originale	Dim. quantizzata	Compressione	F1	ROC-AUC	Target	SRAM	Latenza Wokwi
Logistic Regression	4.44 KB	3.20 KB	1.38x	0.9900	0.9945	Arduino Mega	3 KB	~1.330 μ s
Decision Tree d=5	8.07 KB	4.76 KB	1.69x	0.9943	0.9860	Arduino Mega	5 KB	36–48 μ s

MLP INT8	121.69 KB	13.03 KB	9.34x	0.995 9	0.999 3	ESP3 2-C3	13 KB	~5.250 μs
XGBoost	771.12 KB	369.52 KB	2.09x	0.998 9	1.000 0	ESP3 2-C3	378 KB	N/A
LightGB M	1396.2 4 KB	73.85 KB	18.91x	0.999 2	1.000 0	ESP3 2-C3	76 KB	N/A

Tabella 3.8 – Risultati della quantizzazione post-training e del deployment embedded

La latenza Wokwi è misurata su simulatore per i modelli esportati in codice C via m2cgen (LR, DT) e TFLite Micro (MLP). Per XGBoost e LightGBM il formato binary INT8 richiede un interprete C++ personalizzato non implementato nel presente lavoro (cfr. sezione 4.3); la latenza su hardware reale è pertanto indicata come N/A. Il valore di SRAM stimata corrisponde alla memoria occupata dal modello quantizzato a runtime, esclusi i buffer di inferenza.

I dati della Tabella 3.13 consentono di trarre diverse conclusioni operative. La Logistic Regression è l'unico modello ensemble-alternativo effettivamente deployabile su un microcontrollore di fascia bassa come Arduino Mega 2560 (8 KB SRAM): con soli 985 B di SRAM occupata e una latenza di circa 1.330 μs per inferenza singola, è in grado di classificare circa 750 flussi al secondo, un throughput sufficiente per reti IoT di piccola scala. Il prezzo da pagare è la riduzione dell'F1 a 0.9900 rispetto ai modelli ensemble, che tuttavia in molti scenari operativi rimane ampiamente accettabile.

Il Decision Tree con profondità massima pari a 5 rappresenta un buon compromesso: pur garantendo un F1 di 0.9943, superiore a quello della Logistic Regression, richiede 1.163 B di SRAM e presenta una latenza compresa tra 36 e 48 μs, risultando comunque compatibile con requisiti di quasi real-time in scenari IoT a bassa intensità di traffico. La variazione della latenza (36–48 μs) è legata al diverso percorso che ciascun campione percorre all'interno dell'albero: alcuni flussi, caratterizzati da specifiche configurazioni di feature, raggiungono nodi foglia più profondi, richiedendo quindi un numero maggiore di confronti durante la fase di inferenza.

L'MLP INT8 su ESP32-C3 rappresenta il miglior compromesso tra prestazioni predittive e deployabilità su hardware di fascia intermedia: con F1 = 0.9959 e ROC-AUC = 0.9993, si posiziona tra i modelli boosting e la Logistic Regression in termini di accuratezza, occupando 13.344 B di SRAM — abbondantemente nei limiti dell'ESP32-C3 (400 KB) — con una latenza di circa 5.250 μs per inferenza. La compressione ottenuta dalla quantizzazione INT8 è particolarmente significativa: il modello originale in float32 occupava 121,69 KB, il modello quantizzato riduce questa dimensione a 13,03 KB con un rapporto di compressione di 9,34x,

senza degradazione apprezzabile delle metriche di classificazione rispetto alla versione non quantizzata ($F1 = 0.9959$ vs $F1 = 0.9959$ originale dell'MLP sklearn, ROC-AUC identico a 0.9993).

XGBoost e LightGBM, pur rientrando nei limiti di SRAM dell'ESP32-C3 dopo la quantizzazione (rispettivamente 369 KB e 73,85 KB su un limite di 400 KB), non dispongono di un interprete C++ per il formato binary INT8 prodotto dalla pipeline di esportazione. La misura della latenza su Wokwi non è pertanto disponibile per questi due modelli nel presente lavoro. Tuttavia, considerando che la struttura di XGBoost quantizzato con 369 KB di SRAM occupata si avvicina molto al limite dell'ESP32-C3, in un sistema reale occorrerebbe considerare anche la memoria aggiuntiva richiesta dai buffer di inferenza a runtime. LightGBM, con soli 73,85 KB dopo la compressione 18,91x, risulta il modello con il miglior rapporto tra accuratezza predittiva ($F1 = 0.9992$, ROC-AUC = 1.0000) e ingombro quantizzato tra tutti i modelli ensemble testati.

Dal punto di vista operativo, la selezione del modello da implementare dipende dalle specifiche dell'hardware disponibile e dai requisiti di accuratezza richiesti dal sistema. Nei gateway embedded con vincoli di memoria particolarmente severi (fino a 8 KB di SRAM), le uniche soluzioni realmente utilizzabili sono la Logistic Regression e il Decision Tree con profondità 5. Su hardware di fascia intermedia, come ESP32-C3, la MLP in versione INT8 garantisce un buon compromesso in termini di prestazioni predittive risultando pienamente compatibile secondo le verifiche effettuate in simulazione.

Per deployment su gateway edge dotati di risorse più ampie (fino a circa 400 KB di memoria per il modello), la versione quantizzata di LightGBM costituisce invece la soluzione più vantaggiosa, unendo le migliori prestazioni predittive del gruppo a una riduzione della dimensione pari a 18,91 volte rispetto al modello originale.

4. Conclusioni

4.1 Sintesi critica del lavoro

La tesi ha affrontato il problema dell'Intrusion Detection in ambienti IoT e IIoT attraverso un approccio data-driven basato su Machine Learning supervisionato. Partendo dall'analisi dello stato dell'arte sulle vulnerabilità specifiche di questi ecosistemi e sulle tecniche IDS esistenti, è stata progettata e implementata una pipeline sperimentale completa, modulare e riproducibile per la classificazione del traffico di rete rappresentato tramite network flows. Il problema è stato

formalizzato sia come classificazione binaria sia come classificazione multiclass, consentendo di valutare il sistema a due livelli di difesa complementari.

I risultati sperimentali prodotti dalla combinazione tra network flows e modelli ensemble supervisionati dimostrano prestazioni molto elevate nel task binario. LightGBM ottiene il miglior profilo predittivo, questo significa che in caso di un flusso malevolo il sistema lo identifica correttamente nel 99.95% dei casi; XGBoost si distingue per il miglior equilibrio tra accuratezza e costo operativo qualificandosi come la scelta più adatta per deployment su hardware edge con vincoli di memoria. Nel task multiclass, XGBoost emerge come il modello più equilibrato grazie al Macro-F1 più alto e a un profilo computazionale favorevole. La classe MITM, con soli 1.041 campioni e caratteristiche di traffico ambigue, si conferma la sfida principale dell'intero sistema.

La valutazione cross-domain effettuata sul dataset CIC-IoT-2023 ha consentito di misurare il generalisation gap dei modelli, confermando al contempo la buona portabilità del framework proposto. L'analisi SHAP ha permesso inoltre di identificare le feature maggiormente influenti nella classificazione della classe MITM, fornendo spiegazioni sia locali sia globali utili per l'analisi in ambito di sicurezza informatica.

L'adozione della quantizzazione INT8 ha reso possibile il deployment in scenari embedded: Logistic Regression (3.20 KB, $\sim 1.330 \mu s$), Decision Tree con profondità pari a 5 (4.76 KB, $36-48 \mu s$) e MLP in versione INT8 (13.03 KB, $\sim 5.250 \mu s$) sono stati validati con successo tramite il simulatore Wokwi su piattaforme Arduino Mega 2560 ed ESP32-C3.

Dal punto di vista metodologico, il lavoro mette in luce tre considerazioni fondamentali che vanno oltre il caso specifico analizzato. La prima riguarda il ruolo delle metriche di valutazione, che è tanto rilevante quanto la scelta dell'algoritmo: in scenari con class imbalance, l'accuracy può risultare fuorviante, rendendo quindi necessario l'utilizzo sistematico di metriche come Macro-F1 e PR-AUC per ottenere una valutazione più accurata e rappresentativa.

Il secondo punto riguarda invece il peso delle metriche legate all'efficienza operativa: latenza e dimensione del modello devono essere considerate componenti essenziali nella valutazione di un IDS, poiché un incremento marginale dell'accuratezza accompagnato da un forte aumento di costi computazionali o memoria può rendere il modello meno adatto all'impiego in contesti reali.

Il terzo punto evidenzia che l'analisi a livello di singola classe non rappresenta un elemento accessorio, ma un requisito essenziale poiché le classi meno frequenti e più critiche necessitano infatti di un'attenzione dedicata, che le metriche aggregate tendono a mascherare. Inoltre, la quantizzazione INT8 post-training si dimostra applicabile anche agli ensemble: ad

esempio, LightGBM passa da 1.396 KB a 73.85 KB (riduzione di 18.91x) mantenendo un F1 pari a 0.9992, rendendo possibile il deployment su hardware con vincoli di SRAM.

Il contributo principale della tesi risiede nella costruzione di un framework metodologico rigoroso, trasparente e trasferibile. Questo framework riesce a dimostrare che la qualità di un sistema IDS basato su ML dipende principalmente da un insieme integrato di fattori composto da qualità e preprocessing dei dati, scelta delle metriche adeguate al contesto sbilanciato, analisi per singola classe in aggiunta alle metriche aggregate e valutazione congiunta di prestazioni predittive ed efficienza operativa. L'adozione di questo approccio sistematico piuttosto che la sola ricerca del modello con accuracy più alta, produce sistemi più affidabili, più onesti nella comunicazione dei limiti e più vicini alle esigenze di un impiego reale in ambienti IoT e IIoT.

4.2 Limiti del lavoro

Il primo e più rilevante limite riguarda la natura offline della valutazione. Gli esperimenti sono condotti su un dataset statico e già etichettato, in condizioni controllate che non replicano la complessità di un ambiente operativo reale. In particolare, non viene affrontato il problema del concept drift, cioè la variazione nel tempo della distribuzione del traffico dovuta a cambiamenti nei dispositivi, nelle applicazioni o nelle tattiche degli attaccanti, che in ambienti IoT/IIoT reali può rendere rapidamente obsoleto un modello addestrato su dati storici. Un IDS operativo richiederebbe meccanismi di monitoraggio continuo delle performance e di riaddestramento periodico del modello, aspetti che esulano dallo scope sperimentale di questa tesi.

Un secondo limite è rappresentato dall'assenza di un processo sistematico di tuning degli iperparametri, dal momento che i modelli sono stati configurati sulla base di impostazioni riportate in letteratura. Nel caso di XGBoost e LightGBM, parametri come *learning_rate*, *max_depth*, *num_leaves* e i termini di regolarizzazione incidono in modo significativo sulle prestazioni, in particolare in presenza di class imbalance, dove una loro ottimizzazione mirata potrebbe determinare miglioramenti non trascurabili del Macro-F1 nel task multiclass.

Il terzo limite riguarda la gestione attiva dello sbilanciamento. Nonostante il problema dello sbilanciamento sia documentato in dettaglio, nel task principale su TON_IoT i modelli sono stati addestrati nella configurazione standard senza strategie esplicite di bilanciamento. Solo nella validazione cross-dataset (sezione 3.10) è stato adottato `class_weight='balanced'`, mostrando che questa scelta influisce sul generalisation gap. Questa scelta consente di valutare le prestazioni dei modelli nella loro configurazione standard, ma ha un impatto diretto sul recall delle classi minoritarie. In un contesto operativo di sicurezza, dove la rilevazione di un attacco

MITM, seppur raro, può essere critica, questa limitazione rappresenta la lacuna più rilevante dal punto di vista pratico.

4.3 Sviluppi futuri

Gli sviluppi futuri si articolano su quattro livelli complementari, ovvero metodologico, di interpretabilità, architetturale e di deployment embedded.

Livello metodologico. La priorità più urgente sul piano metodologico è l'estensione della cross-validation k-fold al task multiclass. Nel presente lavoro la validazione con StratifiedKFold a 5 fold è stata condotta sul task binario (sezione 3.8), confermando la stabilità statistica dei risultati degli ensemble con deviazioni standard inferiori a 0.005 su tutte le metriche principali.

Un'estensione strutturata al task multiclass consentirebbe di valutare la variabilità campionaria del Macro-F1 per ogni classe, in particolare per MITM, dove la scarsità di esempi rende le stime basate su un singolo split meno robuste. In parallelo, si potrebbe introdurre l'uso di test statistici formali, come il Wilcoxon signed-rank test e il test di McNemar, per effettuare confronti rigorosi tra i modelli, superando così l'approccio esclusivamente descrittivo adottato in questo studio.

Sul piano della gestione dello sbilanciamento delle classi, nel task multiclass su TON_IoT i modelli sono stati valutati in configurazione standard senza strategie esplicite di bilanciamento, al fine di produrre una baseline confrontabile con la letteratura. Nella validazione cross-dataset (sezione 3.10) è stato applicato `class_weight=balanced`, mostrando che questa scelta influisce sul generalisation gap. Un'adozione sistematica di SMOTENC — la variante di SMOTE progettata per dati misti numerici e categorici — applicata nello spazio delle feature prima della trasformazione OHE, potrebbe migliorare significativamente il recall sulla classe MITM nel task multiclass, che nel presente lavoro rappresenta il principale punto di debolezza del sistema.

L'ottimizzazione sistematica degli iperparametri costituisce un'ulteriore area di miglioramento non ancora approfondita. I modelli sono stati infatti impostati utilizzando configurazioni derivate dalle best practice presenti in letteratura, ad eccezione dei parametri legati alla quantizzazione, `tree_method=hist` e `max_bin=256` per XGBoost, `max_bin=255` per LightGBM, definiti appositamente in funzione del deployment embedded. L'impiego di tecniche come Random Search o Bayesian Optimization, concentrate sui parametri più rilevanti, come `learning_rate`, `max_depth` e `num_leaves`, potrebbe portare a miglioramenti significativi nel task multiclass, dove la maggiore complessità rende i modelli particolarmente sensibili alla scelta degli iperparametri. L'utilizzo di ensemble di secondo livello, come stacking o blending tra XGBoost, LightGBM e

Random Forest all'interno di un metaclassificatore, rappresenta un'ulteriore direzione metodologica in grado di rafforzare i vantaggi già offerti dai singoli modelli.

Livello di generalizzabilità cross-dataset. La validazione sul CIC-IoT-Dataset2023 ha evidenziato che il framework può essere applicato anche a traffico proveniente da ambienti diversi rispetto a quello di training grazie alla feature engineering unificata. Allo stesso tempo, il generalisation gap osservato mostra una riduzione misurabile delle prestazioni, riconducibile principalmente alle differenze distributive tra i due contesti di raccolta dati.

In prospettiva, ulteriori sviluppi includono la sperimentazione su dataset caratterizzati da una maggiore distanza strutturale, come UNSW-NB15, oppure su dati provenienti da ambienti industriali SCADA, con l'obiettivo di verificare la robustezza del framework in presenza di domain shift più accentuati.

Una direzione di ricerca particolarmente promettente è l'utilizzo di tecniche di domain adaptation, come la riduzione della Maximum Mean Discrepancy o metodi basati su apprendimento avversariale, finalizzate a diminuire la distanza tra le distribuzioni del dominio sorgente e di quello target. Il vantaggio principale è la possibilità di mitigare il domain shift senza richiedere dati etichettati nel dominio di destinazione.

Livello di deployment embedded. Il lavoro ha dimostrato che la quantizzazione INT8 post-training consente di ridurre significativamente le dimensioni dei modelli mantenendo metriche di classificazione elevate (sezione 3.13). Tuttavia il formato binario prodotto da `embedded_model_io` richiede la scrittura di un interprete C++ per l'inferenza diretta su microcontrollore, componente non inclusa nel presente lavoro. Lo sviluppo di un interprete leggero per XGBoost e LightGBM in C++ puro — analogo a quello già disponibile per Logistic Regression e Decision Tree tramite `m2cgen` — consentirebbe il test diretto su hardware reale invece che su simulatore Wokwi, producendo misure di latenza e consumo energetico fisicamente accurate. Parallelamente, la riduzione del numero di alberi — `n_estimators` nell'intervallo 50-100 invece degli attuali 300-400 — con un tuning dedicato al contesto embedded consentirebbe di produrre modelli XGBoost e LightGBM direttamente esportabili tramite `m2cgen` in codice C compilabile, abbattendo la dimensione del file a meno di 100 KB e rendendo possibile il deployment su gateway IoT con memoria SRAM limitata.

Livello architetturale. La trasformazione del prototipo verso una piattaforma di monitoraggio operativa richiederebbe l'integrazione con un motore di acquisizione dei flussi in tempo reale, compatibile con strumenti come Zeek o Argus, un modulo di preprocessing in streaming capace

di operare con latenze compatibili con il traffico IoT, e un meccanismo di alerting con soglie adattive e gestione delle priorità basata sulla severità della classe rilevata. In quest'ottica, l'architettura a cascata discussa nella sezione 4.4 — modello binario come primo filtro rapido, modello multiclass come secondo livello diagnostico — si presta naturalmente a un'implementazione distribuita in cui il primo livello viene eseguito direttamente sul gateway embedded e il secondo su un edge server con risorse computazionali adeguate. Il monitoraggio continuo del concept drift, attraverso test statistici sulle distribuzioni delle feature in finestre temporali successive, rappresenta infine un requisito indispensabile per mantenere le prestazioni del sistema nel tempo in presenza di traffico reale evolutivo.

4.4 Implicazioni applicative

I risultati indicano che un IDS prototipale basato su Machine Learning e network flows rappresenta una base solida per lo sviluppo di sistemi di monitoraggio della sicurezza in ambienti IoT e IIoT.

La struttura modulare della pipeline permette di adattarla facilmente a nuovi dataset e contesti operativi senza la necessità di intervenire sull'architettura principale, tramite l'adozione di modelli supervisionati su dati tabellari semplifica l'implementazione e facilita l'integrazione con infrastrutture di sicurezza già esistenti, come piattaforme SIEM quali Splunk, IBM QRadar o Microsoft Sentinel, che generalmente gestiscono eventi in formato strutturato e tabellare.

Dal punto di vista della sicurezza operativa, il sistema proposto può integrarsi in una difesa a strati dove il modello binario funzionando come primo livello di triage rapido classifica ogni flusso come normale o sospetto con latenza sotto i 100 ms per gli ensemble, consentendo un filtraggio in quasi-real-time del traffico. In seguito il modello multiclass costituisce il secondo livello di analisi applicato ai flussi già identificati come sospetti e classifica la tipologia di attacco orientando la risposta dell'operatore di sicurezza. Questa architettura a cascata permette di bilanciare l'efficienza, poiché abbiamo il primo livello che elabora tutto il traffico e il secondo solo quello sospetto, e la profondità di analisi, dato che il secondo livello fornisce informazioni più dettagliate necessarie per la risposta adeguata.

Dal punto di vista industriale, la crescente adozione di infrastrutture IoT e IIoT sta incrementando la richiesta di soluzioni avanzate per la sicurezza delle reti in settori chiave come manifatturiero, energetico, sanitario e dei trasporti. Secondo le stime di mercato, il numero di dispositivi IoT connessi supererà i 30 miliardi entro la fine del decennio (Statista 2023), espandendo proporzionalmente la superficie di attacco che i sistemi IDS devono monitorare. In ambito manifatturiero, la protezione dei sistemi SCADA e

PLC da attacchi come quelli che hanno colpito impianti industriali critici negli ultimi anni rende i sistemi IDS basati su ML una componente sempre più strategica. In ambito sanitario, la protezione dei dispositivi medicali connessi come pacemaker, monitor di glicemia, pompe di infusione da accessi non autorizzati è diventata una priorità regolamentare oltre che operativa.

Dal punto di vista professionale, le competenze sviluppate in questo lavoro si collocano nell'intersezione di tre settori in forte crescita, la Data Science applicata alla sicurezza, l'ingegneria dei sistemi IoT e l'analisi del traffico di rete; la convergenza tra queste aree è destinata a rafforzarsi ulteriormente nei prossimi anni, rendendo la formazione interdisciplinare sempre più importante anche in ottica strategica.

In prospettiva, l'integrazione tra Machine Learning e sicurezza informatica è destinata a diventare un pilastro delle architetture di difesa moderne. Poiché l'emergere di attacchi sempre più sofisticati richiederà sistemi IDS più robusti, interpretativi e adattativi. Il lavoro svolto contribuisce a questo percorso fornendo un framework metodologico documentato e riproducibile, offrendo una base affidabile su cui costruire iterazioni successive verso sistemi IDS operativamente maturi.

5. Bibliografia

Libri

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). Sebastopol, CA: O'Reilly Media.

Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Waltham, MA: Morgan Kaufmann.

Hu, F. (2016). *Security and Privacy in Internet of Things (IoT): Models, Algorithms, and Implementations*. Boca Raton, FL: CRC Press.

Jeschke, S., Brecher, C., Meisen, T., Özdemir, D., & Eschert, T. (2017). *Industrial Internet of Things: Cybermanufacturing Systems*. Cham: Springer.

Stallings, W. (2017). *Computer Security: Principles and Practice* (4th ed.). Hoboken, NJ: Pearson.

Stallings, W. (2017). *Network Security Essentials: Applications and Standards* (6th ed.). Hoboken, NJ: Pearson.

Articoli scientifici

Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., & Anwar, A. (2020). TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems. *IEEE Access*, 8, 165130–165150. <https://doi.org/10.1109/ACCESS.2020.3022862>

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (KDD '16), San Francisco, CA, USA, 785–794. <https://doi.org/10.1145/2939672.2939785>

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154. <https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>

Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774. <https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf>

Xu, B., Sun, L., Mao, X., Ding, R., & Liu, C. (2023). IoT Intrusion Detection System Based on Machine Learning. *Electronics*, 12(20), 4289. <https://doi.org/10.3390/electronics12204289>

Dataset e repository

Moustafa, N., & Slay, J. (2020). *TON_IoT Network Dataset*. Kaggle. <https://www.kaggle.com/datasets/arnobbbhowmik/ton-iot-network-dataset>

Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors*, 23(13), 5941. <https://doi.org/10.3390/s23135941>

Kuznetsov, O. (2025). *IoT Device Network Audit — Binary & Multiclass Intrusion Detection System (IDS)*. GitHub. Utilizzato come riferimento di struttura per la pipeline sperimentale.¹

Il repository del relatore ha fornito la struttura di riferimento iniziale per l'organizzazione della pipeline sperimentale. Per il codice sviluppato nel presente lavoro si rimanda al repository del progetto di tesi: <https://github.com/emanuelepiodebernardis/iot-audit>

Normativa

Regolamento (UE) 2024/1689 del Parlamento Europeo e del Consiglio, del 13 giugno 2024, che stabilisce regole armonizzate sull'intelligenza artificiale, GU L 2024/1689.